Architecture 1001:
x86-64 Assembly
32-bit Calling Conventions

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

32-bit Stack Calling Conventions throwback 1

- In 32 bit code, there are many more calling conventions in use
  - “cdecl” (default for most C code)
    - Caller cleans up the stack
  - “stdcall” (for Windows' Win32 APIs)
    - Callee cleans up the stack
- Function parameters are *push*ed onto stack from right to left
  - leftmost parameter (the first function parameter) ends up at the lowest address

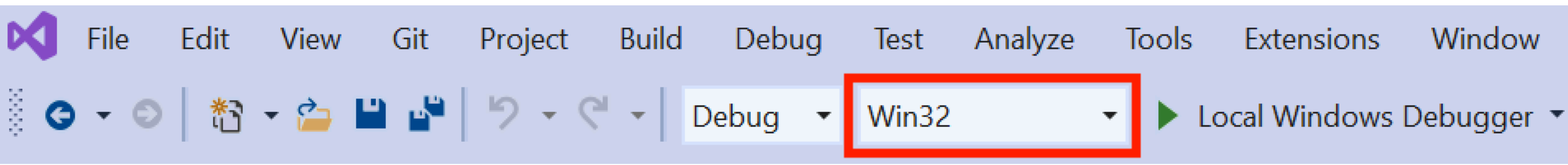
https://docs.microsoft.com/en-us/cpp/cpp/cdecl?view=msvc-160

https://docs.microsoft.com/en-us/cpp/cpp/stdcall?view=msvc-160

32-bit TooManyParameters.c

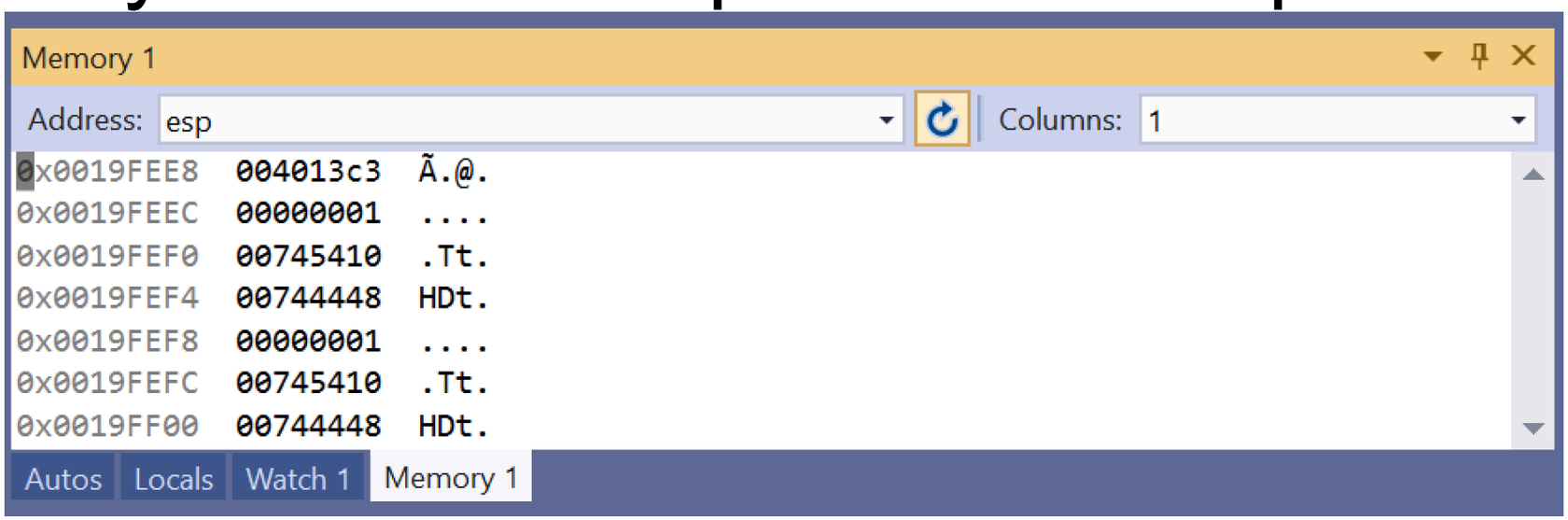
```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

Step 1: Set config to "Win32" instead of "x64" in VS



Step 2: Start debugger

Step 3: Set memory window to esp instead of rsp and 4 bytes at a time





STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

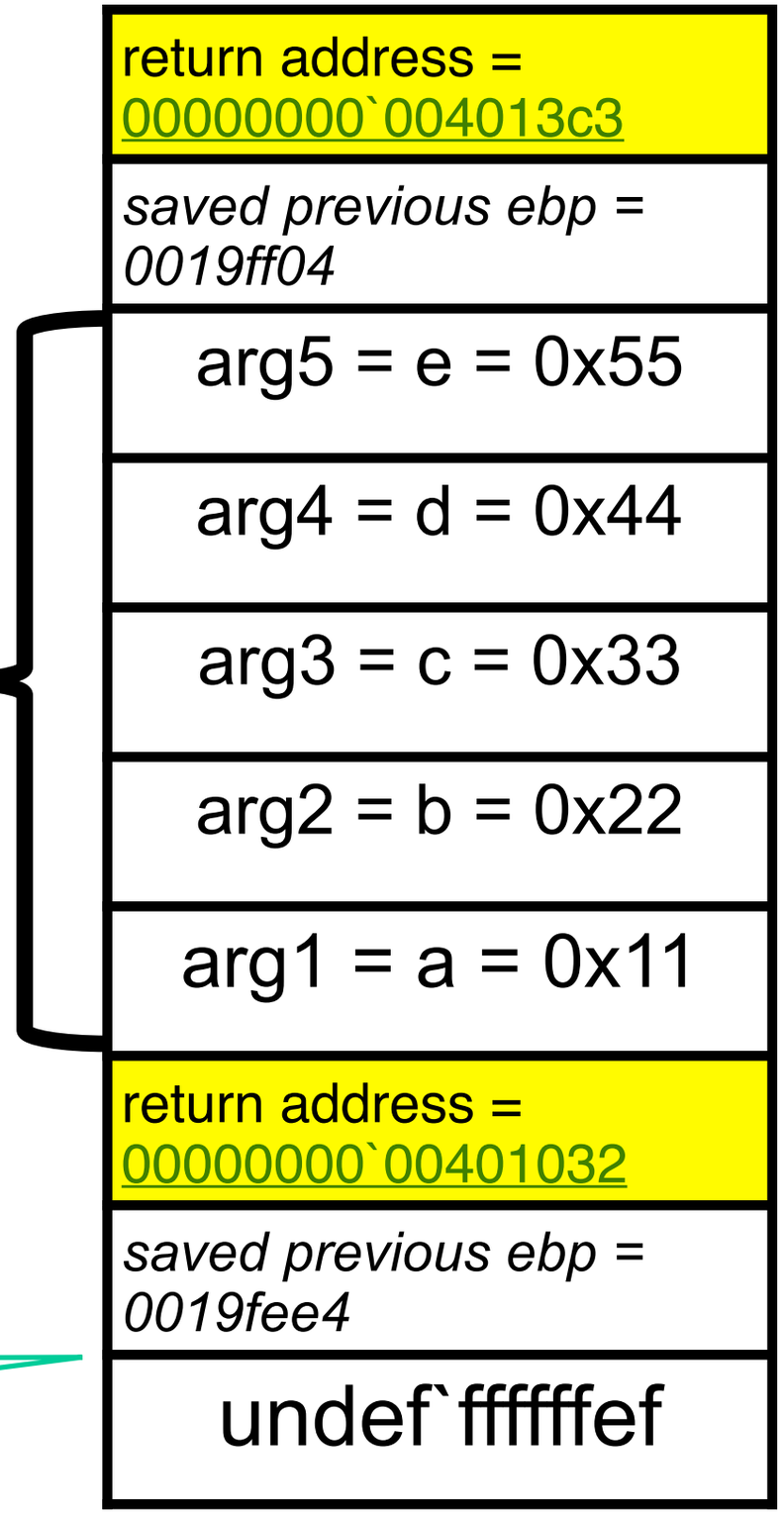
DRAW A STACK DIAGRAM!

32-bit TooManyParameters.c

```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
   int i = a+b-c+d-e;
   return i;
}
int main(){
   return func(0x11,0x22,0x33,0x44,0x55);
}
```

Not Microsoft stack “Shadow Space”, just the normal calling convention!

No alignment attempts
==
no padding

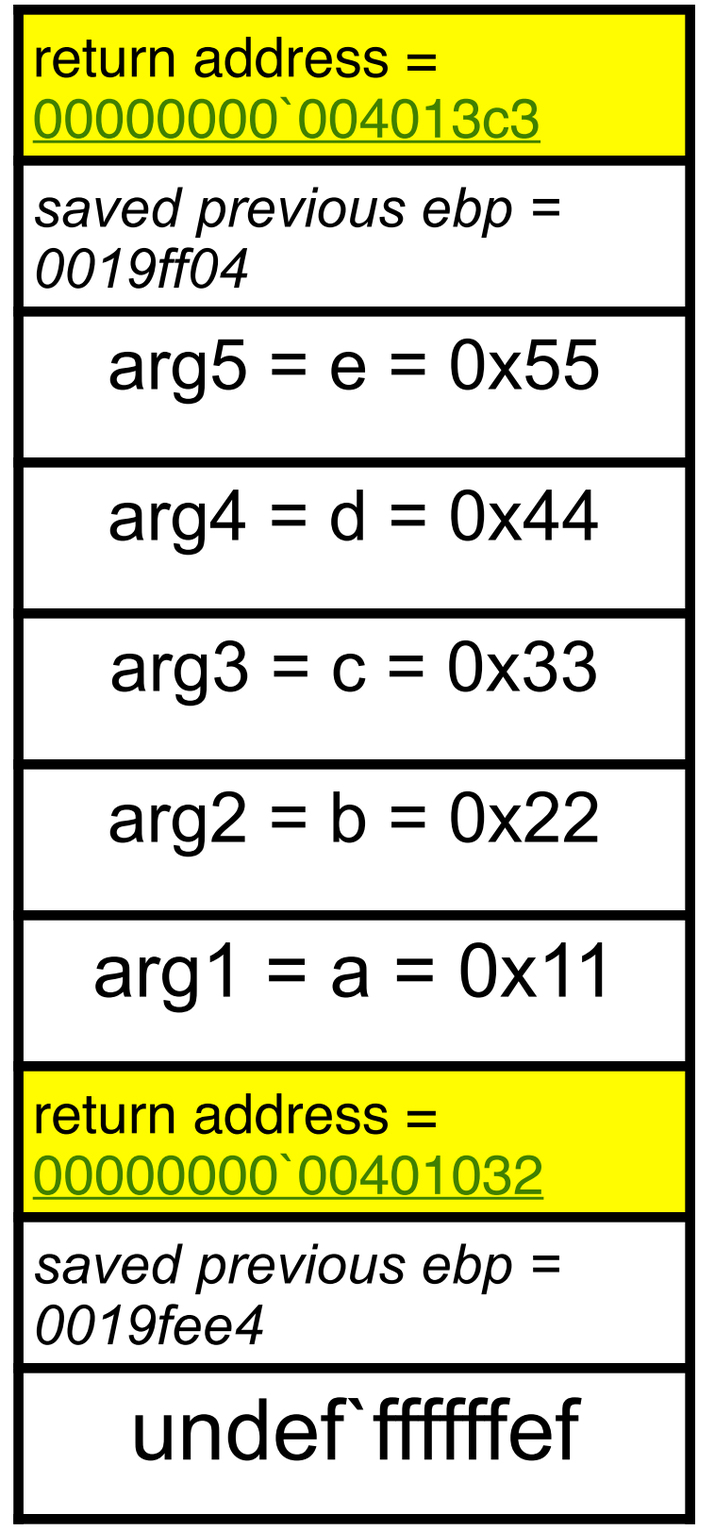


32-bit TooManyParameters.c 2

```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

```
main:
 push        ebp
 mov         ebp,esp
 push        55h
 push        44h
 push        33h
 push        22h
 push        11h
 call        00401000
 add         esp,14h
 pop         ebp
 ret
```

```
func:
 push        ebp
 mov         ebp,esp
 push        ecx
 mov         eax,dword ptr [ebp+8]
 add         eax,dword ptr [ebp+0Ch]
 sub         eax,dword ptr [ebp+10h]
 add         eax,dword ptr [ebp+14h]
 sub         eax,dword ptr [ebp+18h]
 mov         dword ptr [ebp-4],eax
 mov         eax,dword ptr [ebp-4]
 mov         esp,ebp
 pop         ebp
 ret
```



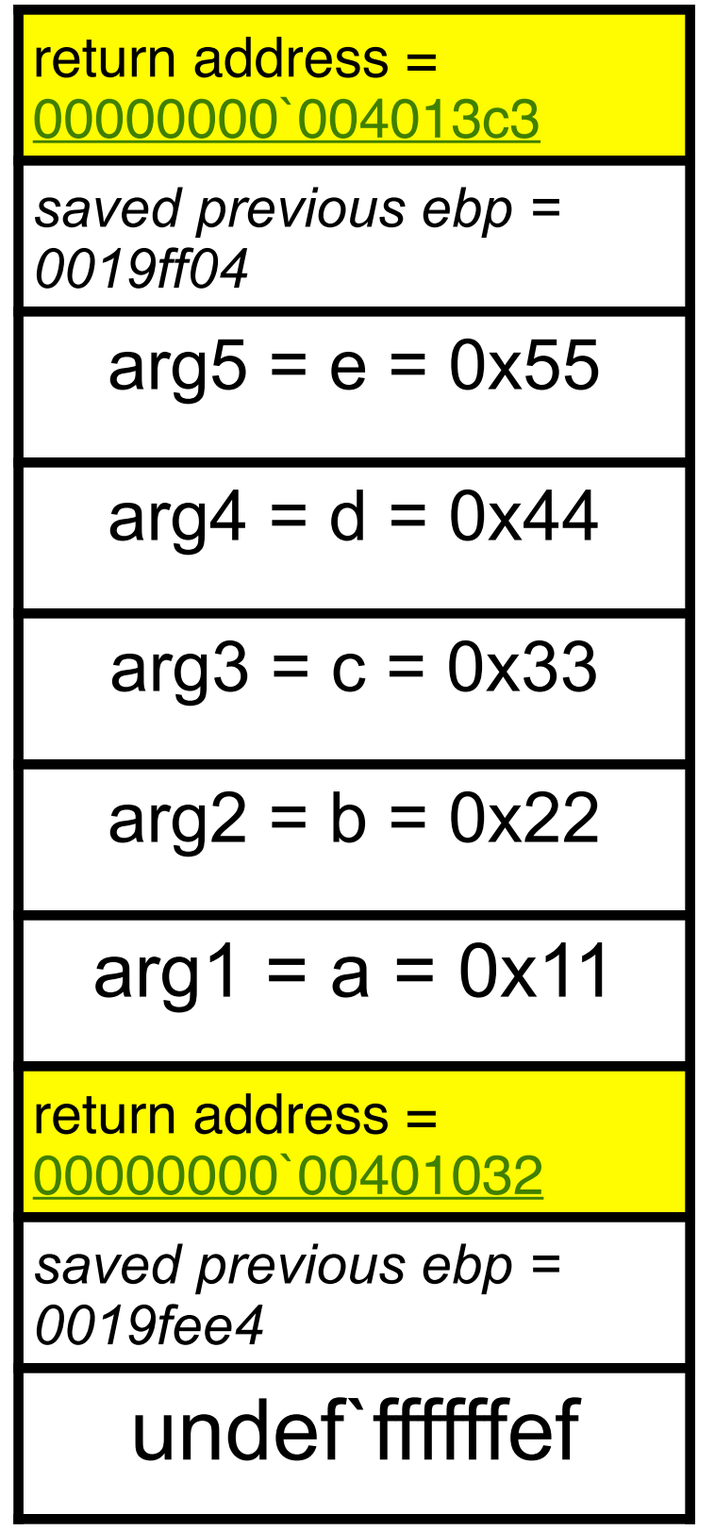
32-bit TooManyParameters.c 2

```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

```
main:
 push        ebp
 mov         ebp,esp
 push        55h
 push        44h
 push        33h
 push        22h
 push        11h
 call        00401000
 add         esp,14h
 pop         ebp
 ret
```

```
func:
                                    ebp+8]
                                    ebp+0Ch]
                                    ebp+10h]
 add         eax,dword ptr [ebp+14h]
 sub         eax,dword ptr [ebp+18h]
 mov         dword ptr [ebp-4],eax
 mov         eax,dword ptr [ebp-4]
 mov         esp,ebp
 pop         ebp
 ret
```

In 32 bit code when you see a bunch of *push*es in front of a *call*, that usually makes it pretty clear what the arguments to the function are

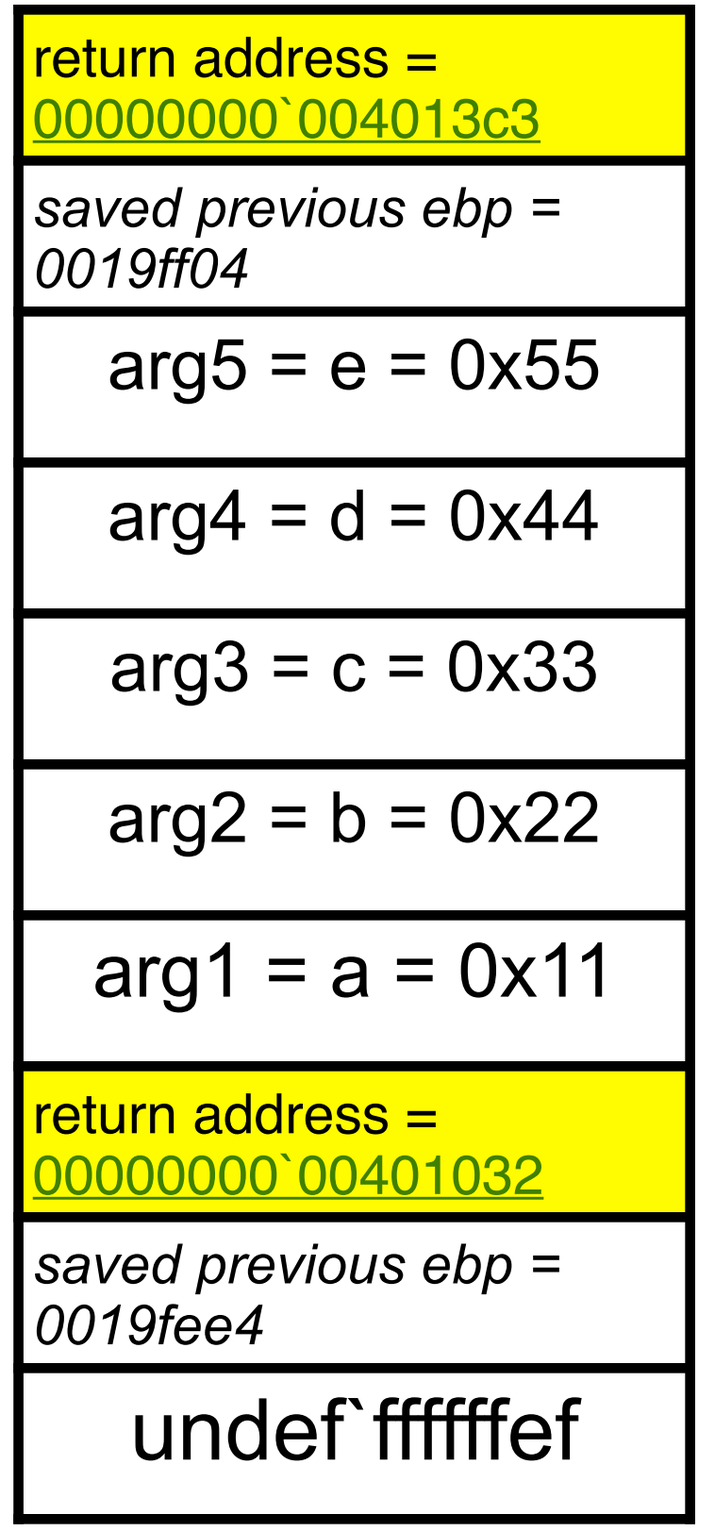


32-bit TooManyParameters.c 2

```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

```
main:
 push        ebp
 mov         ebp,esp
 push        55h
 push        44h
 push        33h
 push        22h
 push        11h
 call        00401000
 add         esp,14h
 pop         ebp
 ret
```

```
func:
 push        ebp
 mov         ebp,esp
 push        ecx
 mov         eax,dword ptr [ebp+8]
 add         eax,dword ptr [ebp+0Ch]
 sub         eax,dword ptr [ebp+10h]
 add         eax,dword ptr [ebp+14h]
 sub         eax,dword ptr [ebp+18h]
 mov         dword ptr [ebp-4],eax
 mov         eax,dword ptr [ebp-4]
 mov         esp,ebp
 pop         ebp
 ret
```



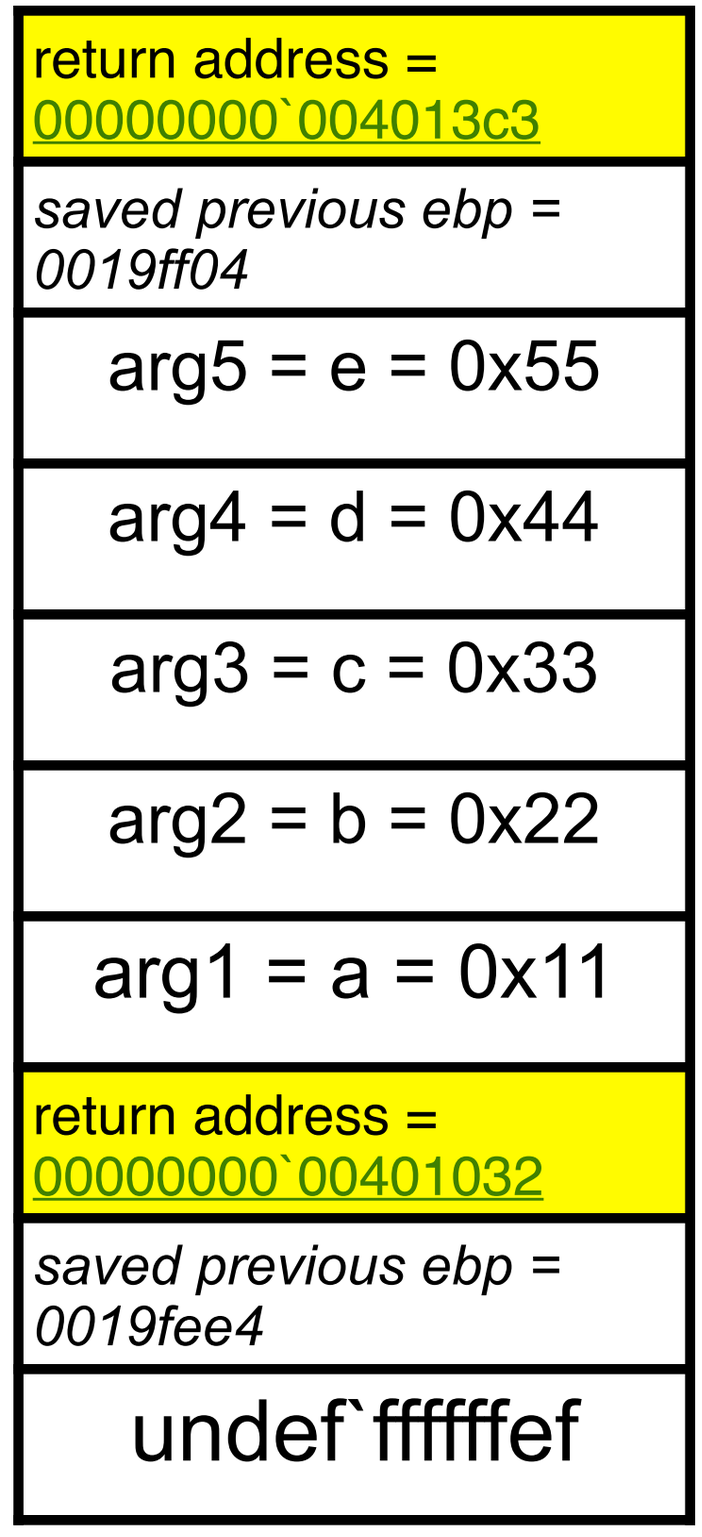
32-bit TooManyParameters.c 2

```
//Change uint64 to int
int func(int a, int b, int c, int d, int e){
  int i = a+b-c+d-e;
  return i;
}
int main(){
  return func(0x11,0x22,0x33,0x44,0x55);
}
```

```
main:
 push        ebp
 mov         ebp,esp
 push        11h
 call        00401000
 add         esp,14h
 pop         ebp
 ret
```

Not a callee-save! It's a VS-ism for allocating a single int worth of space for a local variable!

```
func:
 push        ebp
 mov         ebp,esp
 push        ecx
 mov         eax,dword ptr [ebp+8]
 add         eax,dword ptr [ebp+0Ch]
 sub         eax,dword ptr [ebp+10h]
 add         eax,dword ptr [ebp+14h]
 sub         eax,dword ptr [ebp+18h]
 mov         dword ptr [ebp-4],eax
 mov         eax,dword ptr [ebp-4]
 mov         esp,ebp
 pop         ebp
 ret
```



32-bit VS vs. GCC et al.
Caller vs. Callee-save Regs

VS Caller-saved	EAX		ECX	EDX			
VS Callee-saved		EBX			ESI	EDI	EBP
System V (GCC et al.) Caller-saved	EAX		ECX	EDX	ESI	EDI	
System V (GCC et al.) Callee-saved		EBX					EBP

32-bit Stack Calling Conventions throwback 2

- Both cdecl and stdcall conventions perform explicit *stack frame linkage*
- Each time you enter a new function, the old ebp ("stack frame **b**ase **p**ointer" by Intel convention) gets pushed onto the stack, and the new esp (top-of-the **s**tack **p**ointer, which now points at the copy of ebp), gets *mov*ed into ebp

https://docs.microsoft.com/en-us/cpp/cpp/cdecl?view=msvc-160

https://docs.microsoft.com/en-us/cpp/cpp/stdcall?view=msvc-160

Changing to 32 bit compilation

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

Assembly changes from this:

```
func:
0000000140001000  mov          eax,0BEEFh
0000000140001005  ret
main:
0000000140001010  sub          rsp,28h
0000000140001014  call         func (0140001000h)
0000000140001019  mov          eax,0F00Dh
000000014000101E  add          rsp,28h
0000000140001022  ret
```

Changing to 32 bit compilation

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

Changing to 32 bit compilation

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

Changing to 32 bit compilation

To this:

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000  push          ebp
00401001  mov           ebp,esp
00401003  mov           eax,0BEEFh
00401008  pop           ebp
00401009  ret
main:
00401010  push          ebp
00401011  mov           ebp,esp
00401013  call          func (0401000h)
00401018  mov           eax,0F00Dh
0040101D  pop           ebp
0040101E  ret
```

32-bit Stack Frame Diagram

stack bottom

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

stack top

Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}

func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

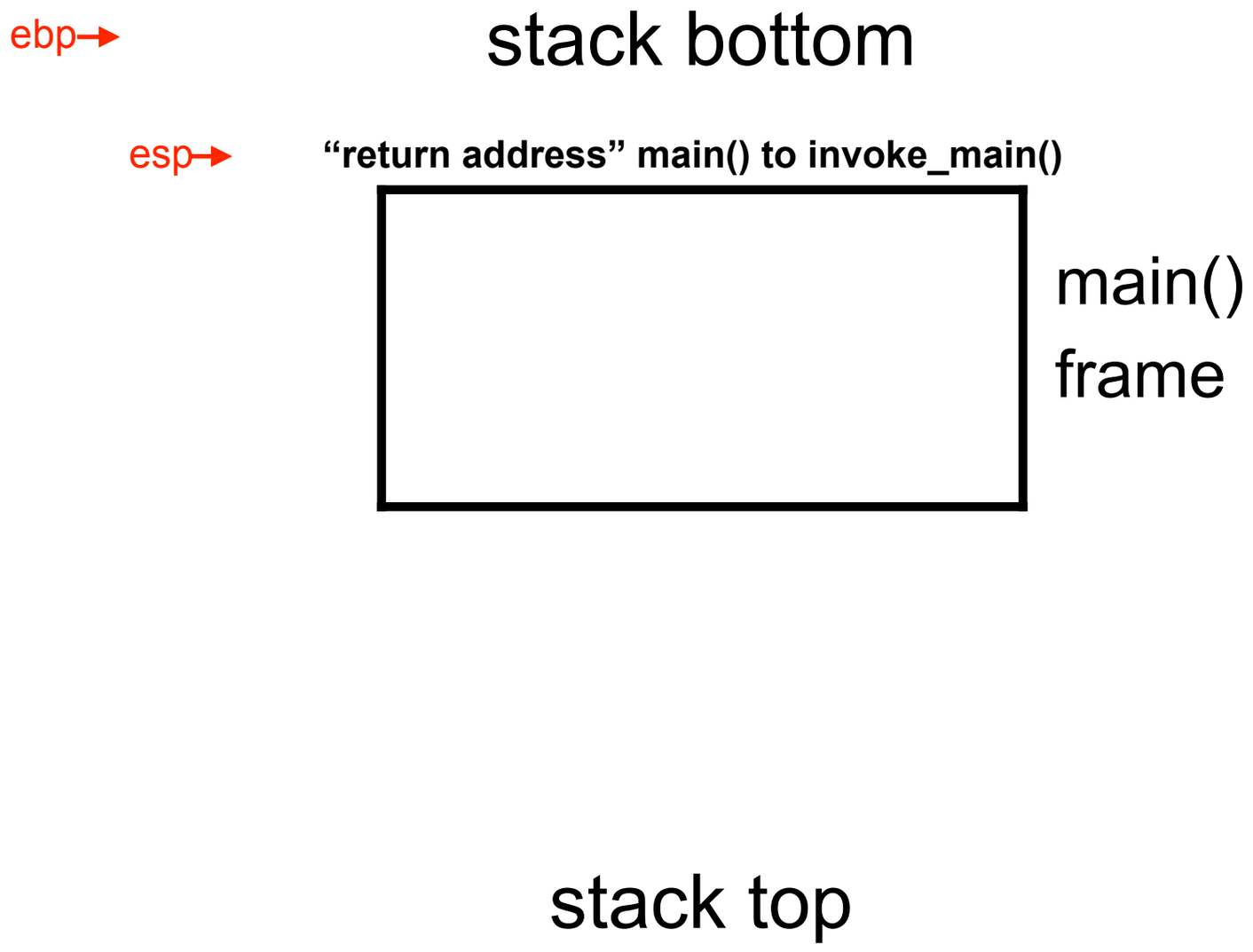


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

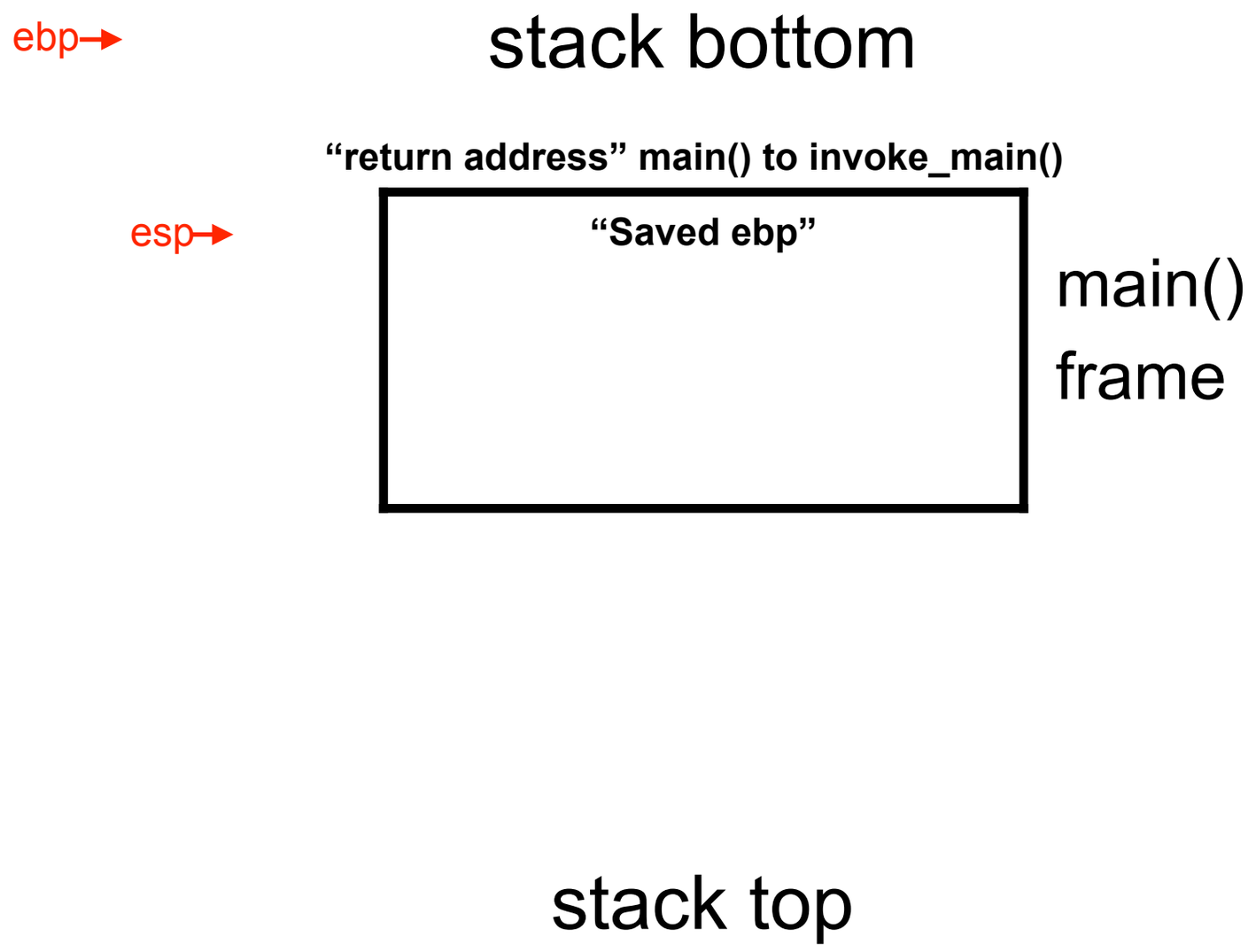


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

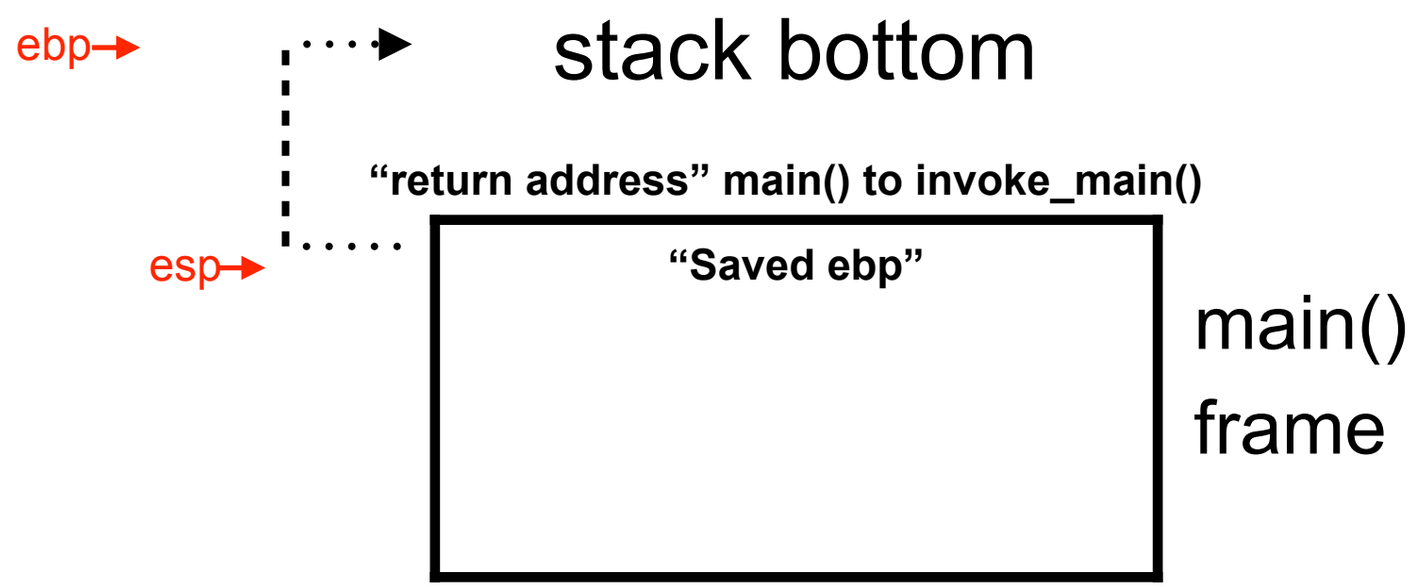


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

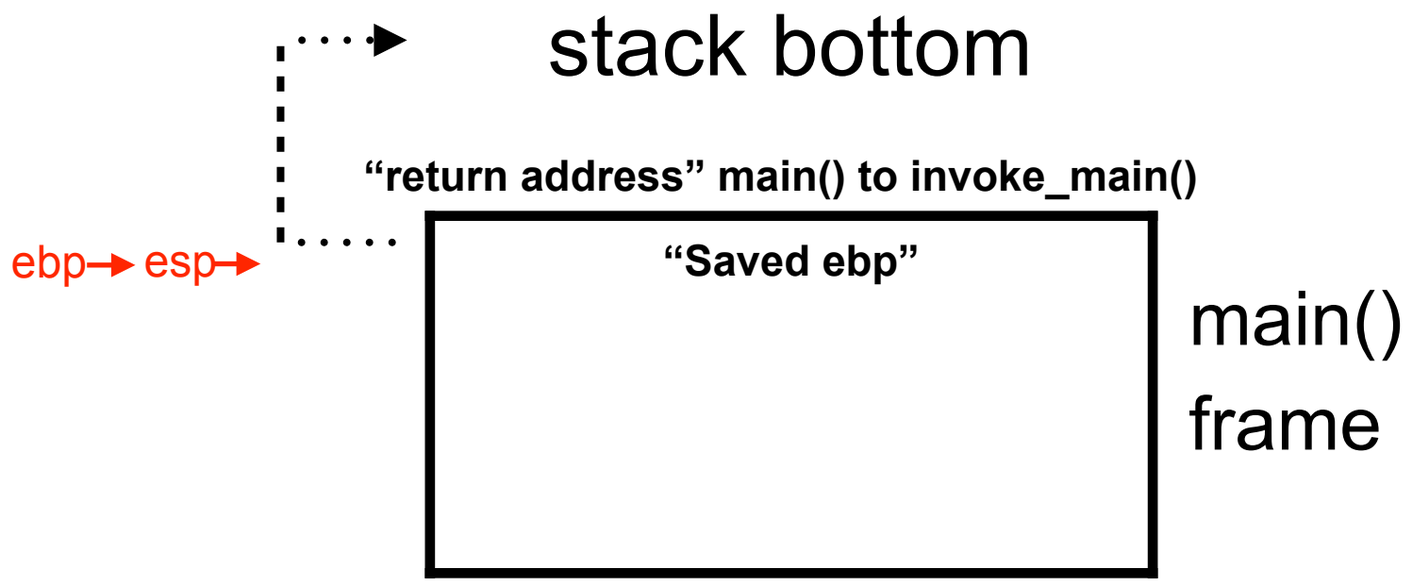
```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```



Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

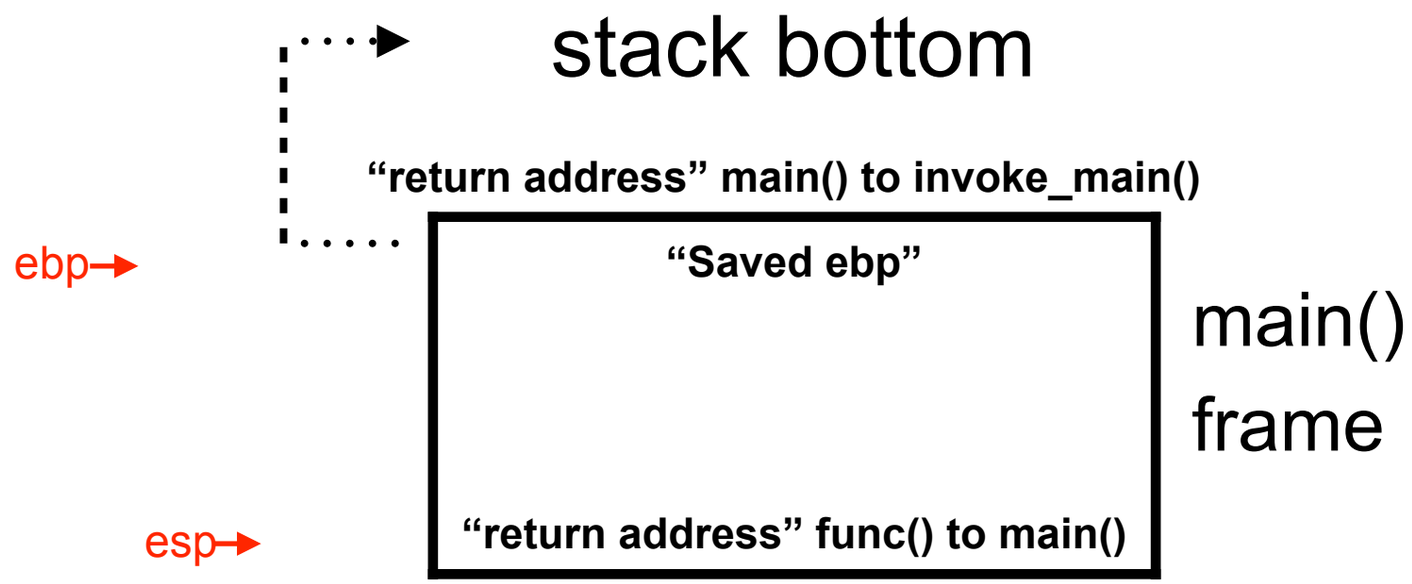



```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```



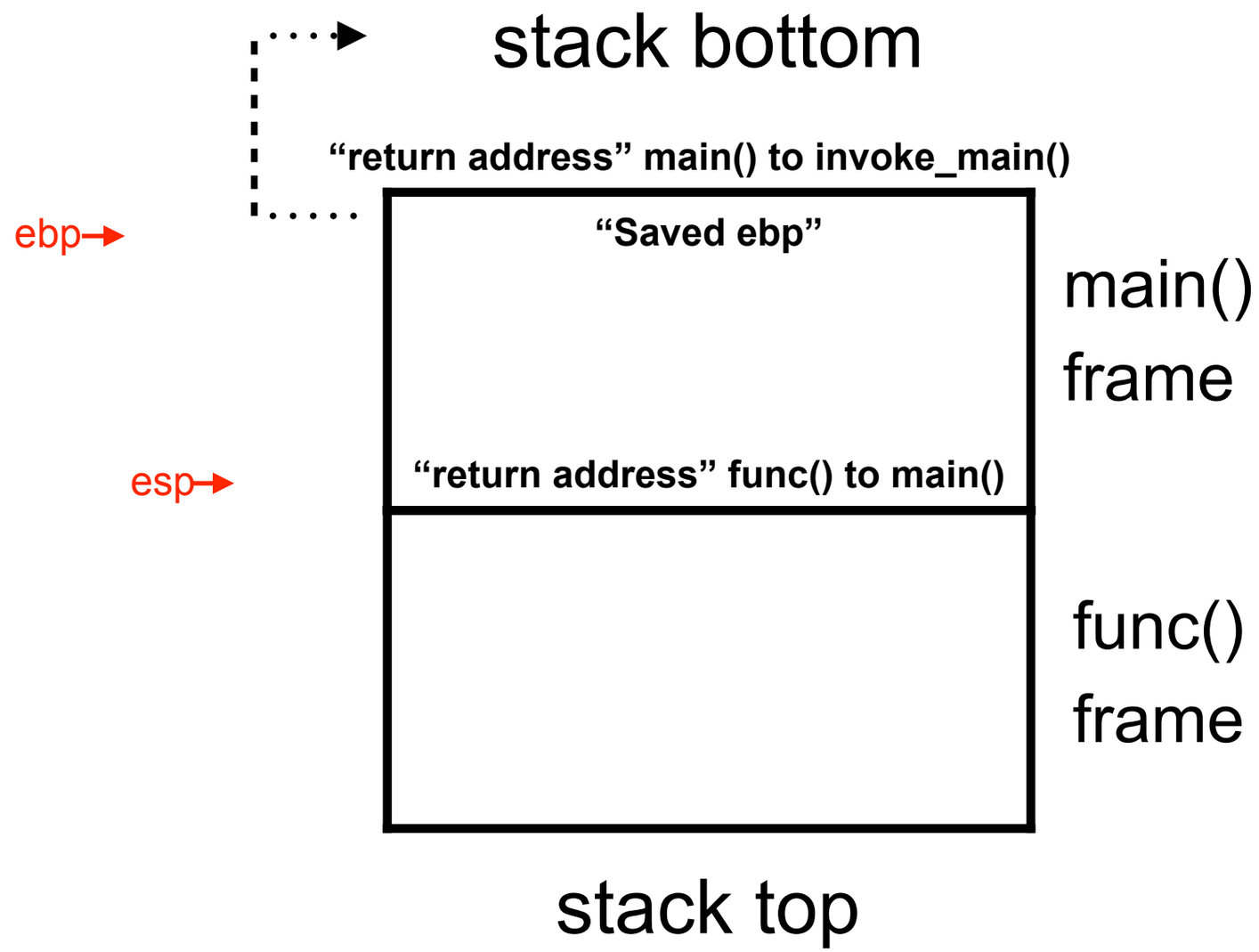

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

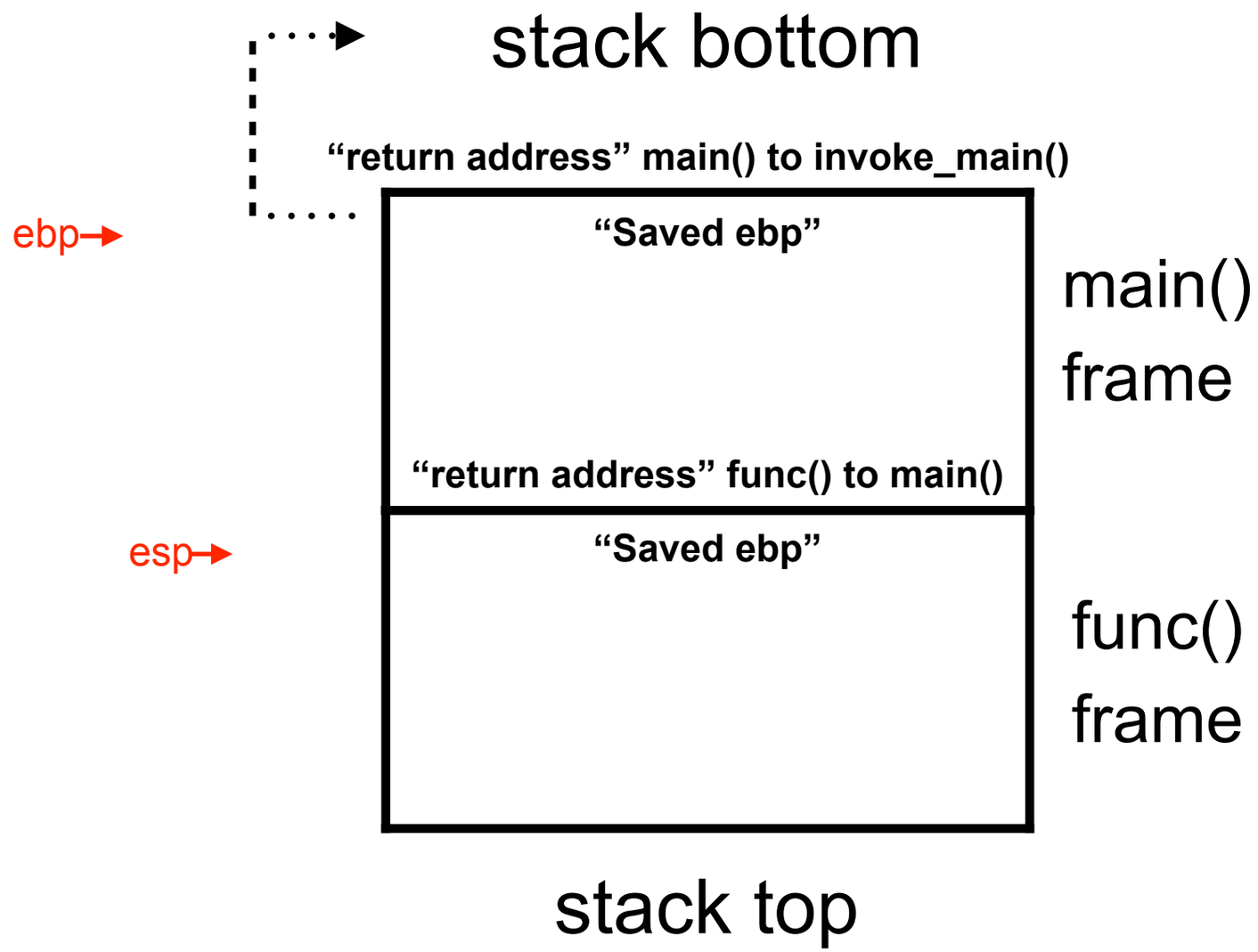


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

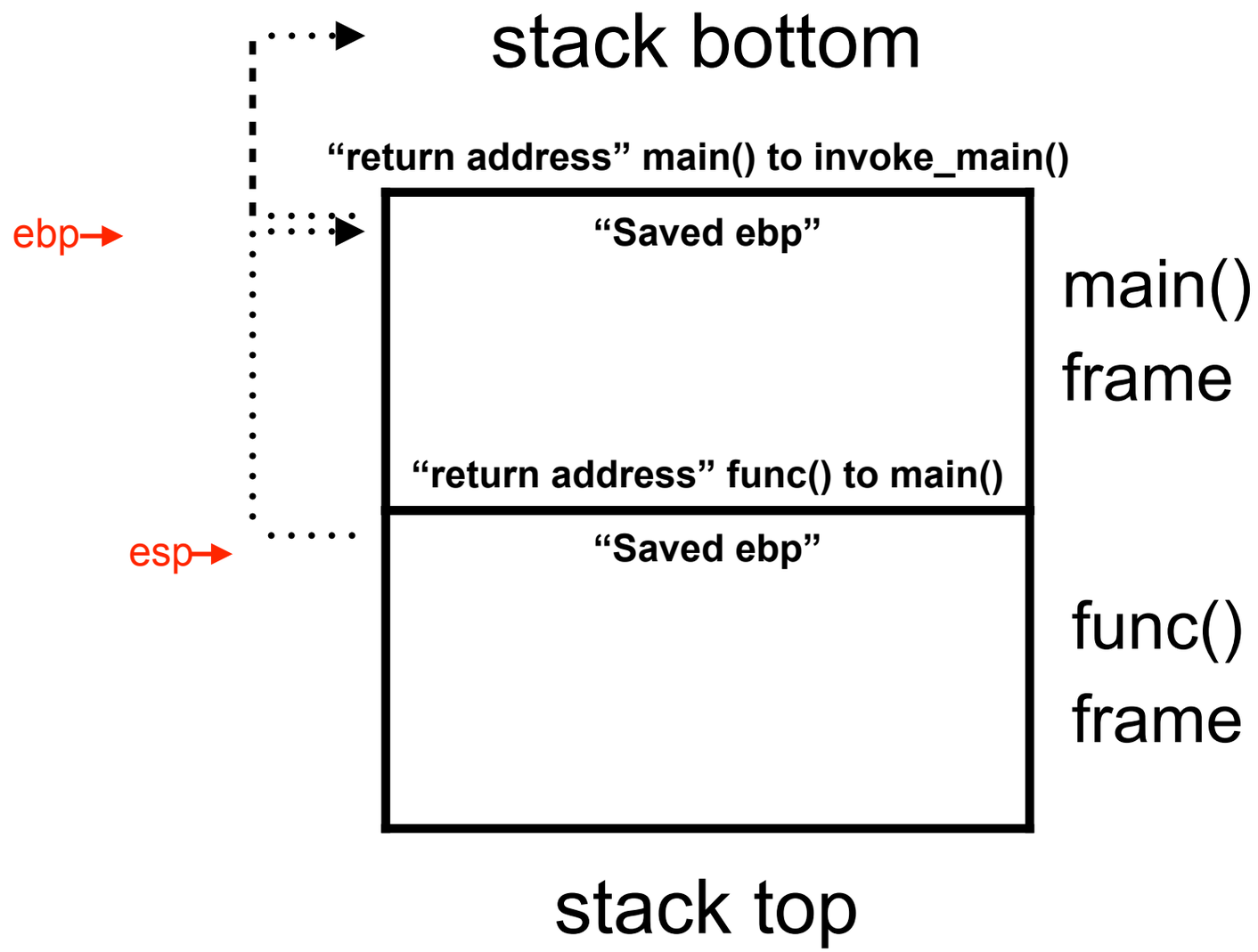


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```

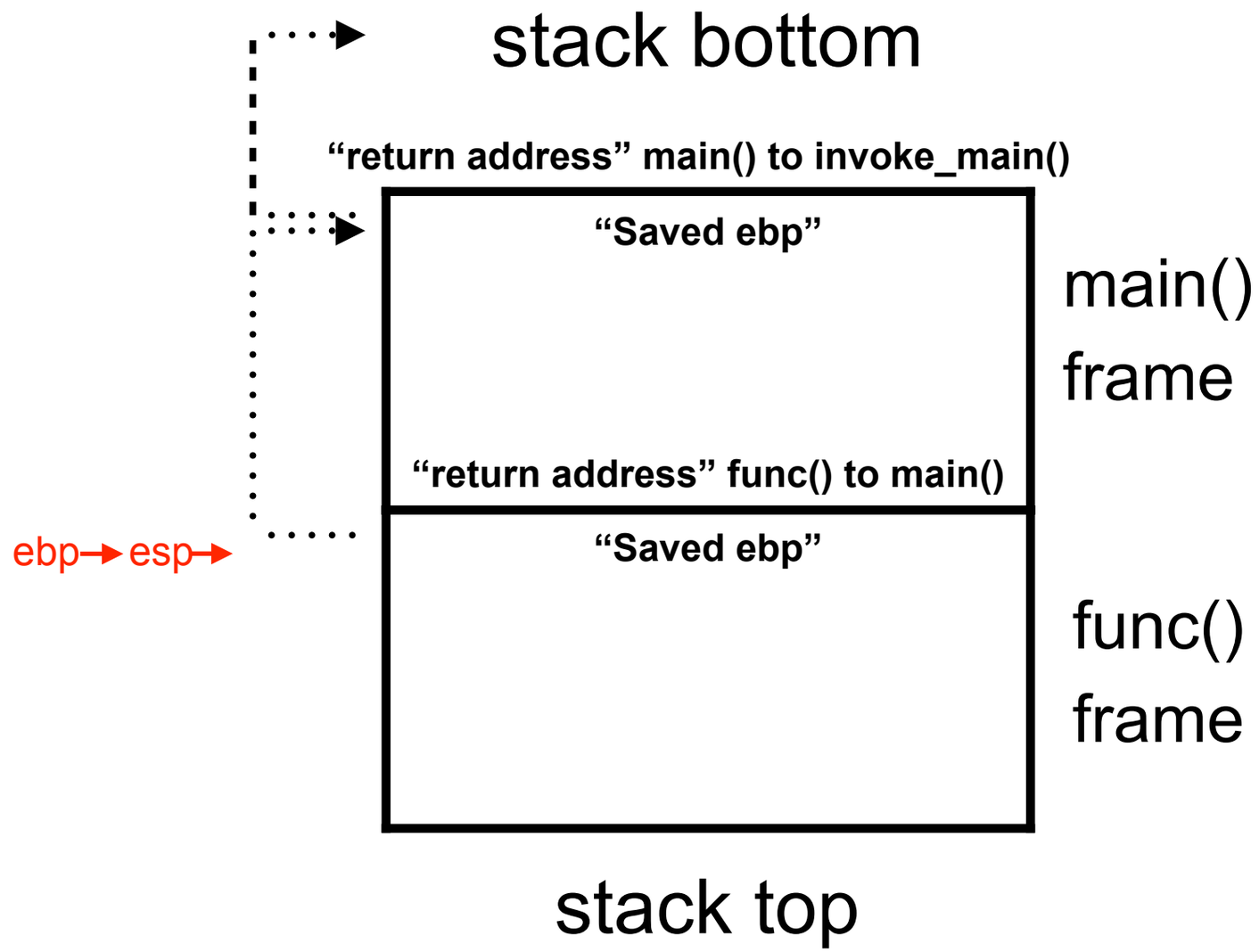


Animated slide, don't let the esp location fool you, it moves!

32-bit Stack Frame Diagram

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
00401000   push          ebp
00401001   mov           ebp,esp
00401003   mov           eax,0BEEFh
00401008   pop           ebp
00401009   ret
main:
00401010   push          ebp
00401011   mov           ebp,esp
00401013   call          func (0401000h)
00401018   mov           eax,0F00Dh
0040101D   pop           ebp
0040101E   ret
```



Animated slide, don't let the esp location fool you, it moves!

32-bit Simple Stack Diagram 4

stack bottom

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

32-bit Simple Stack Diagram 4

ebp→

stack bottom

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

32-bit Simple Stack Diagram 4

ebp→

stack bottom

“return address” main() to invoke_main()

```
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

32-bit Simple Stack Diagram 4

ebp→ stack bottom

esp→ **“return address” main() to invoke_main()**

```
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

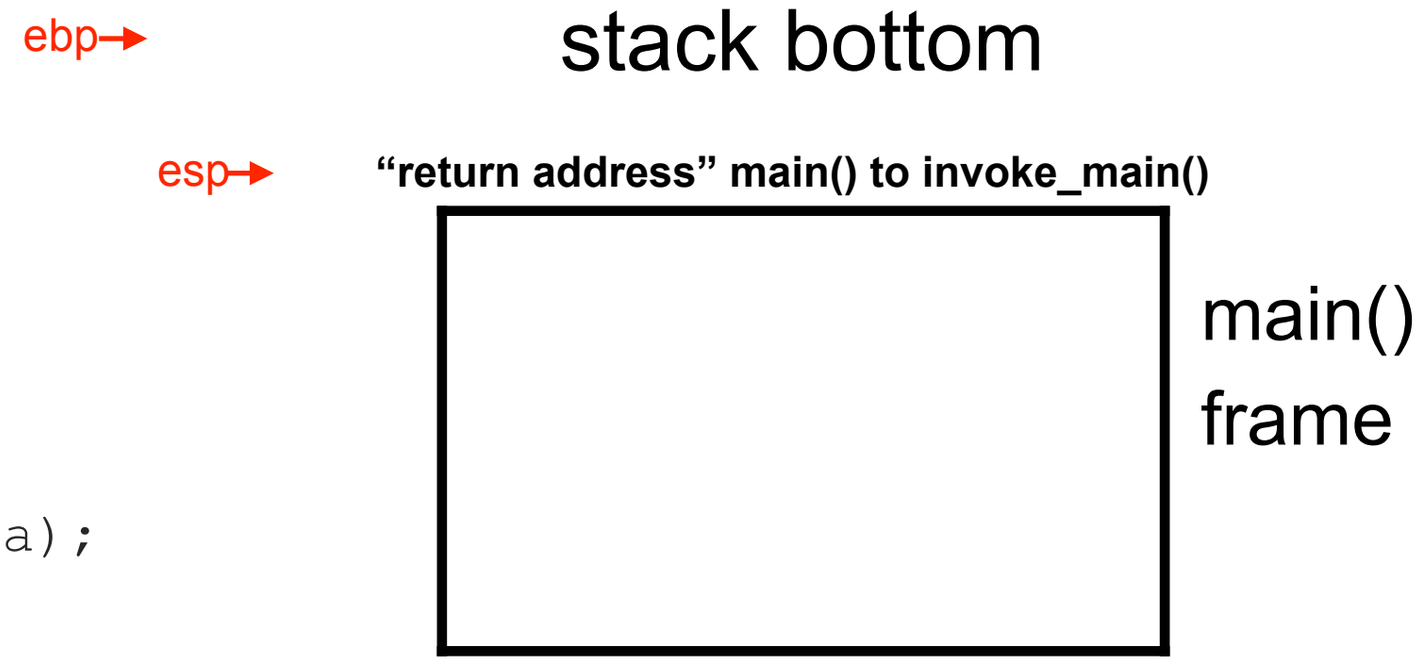
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



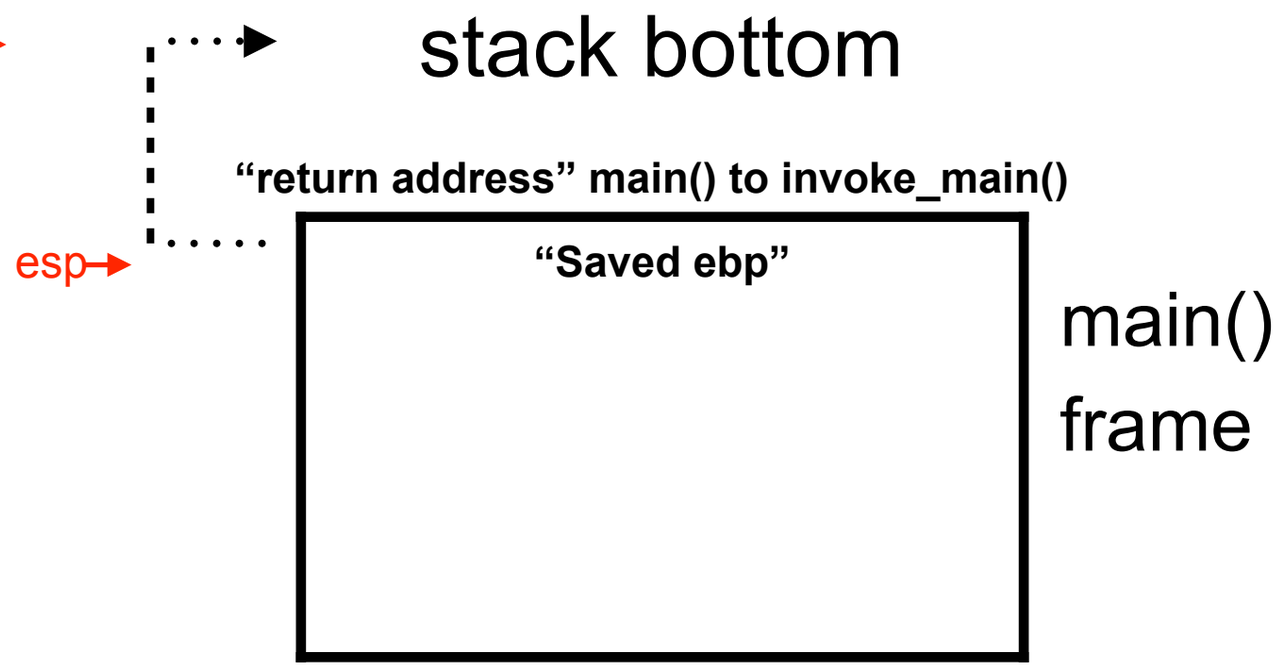
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



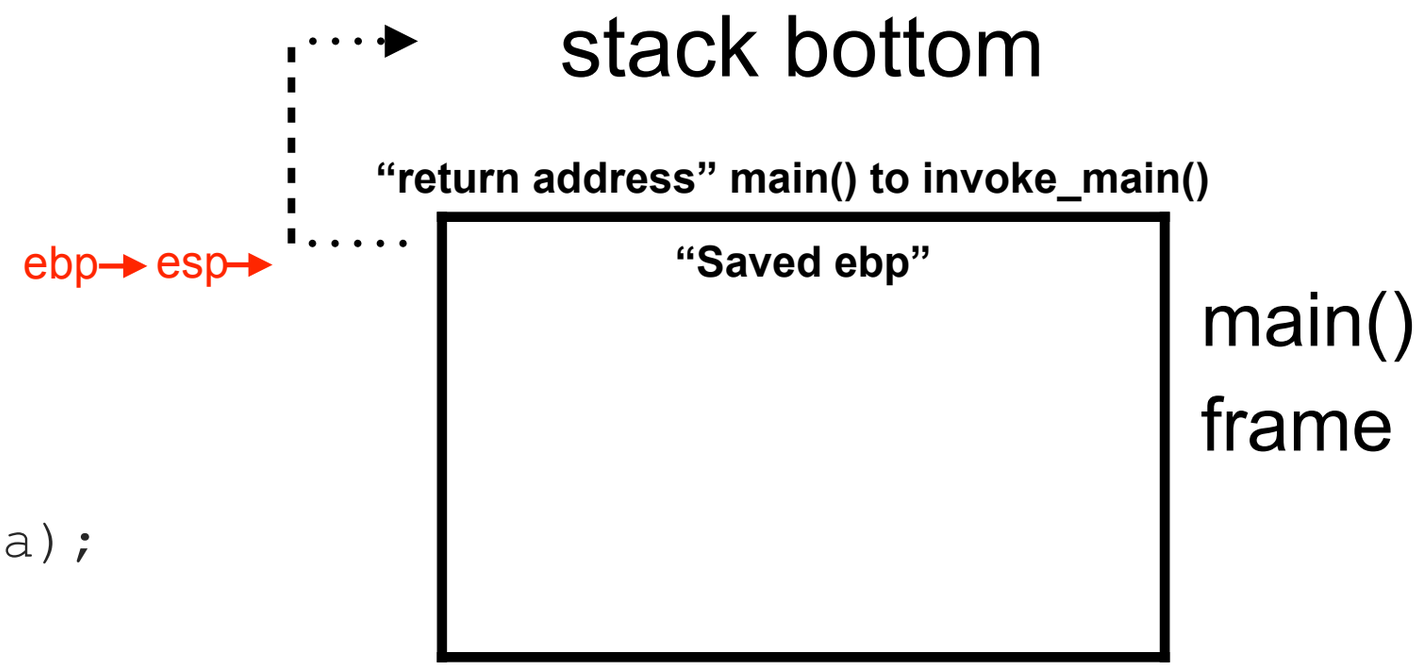
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



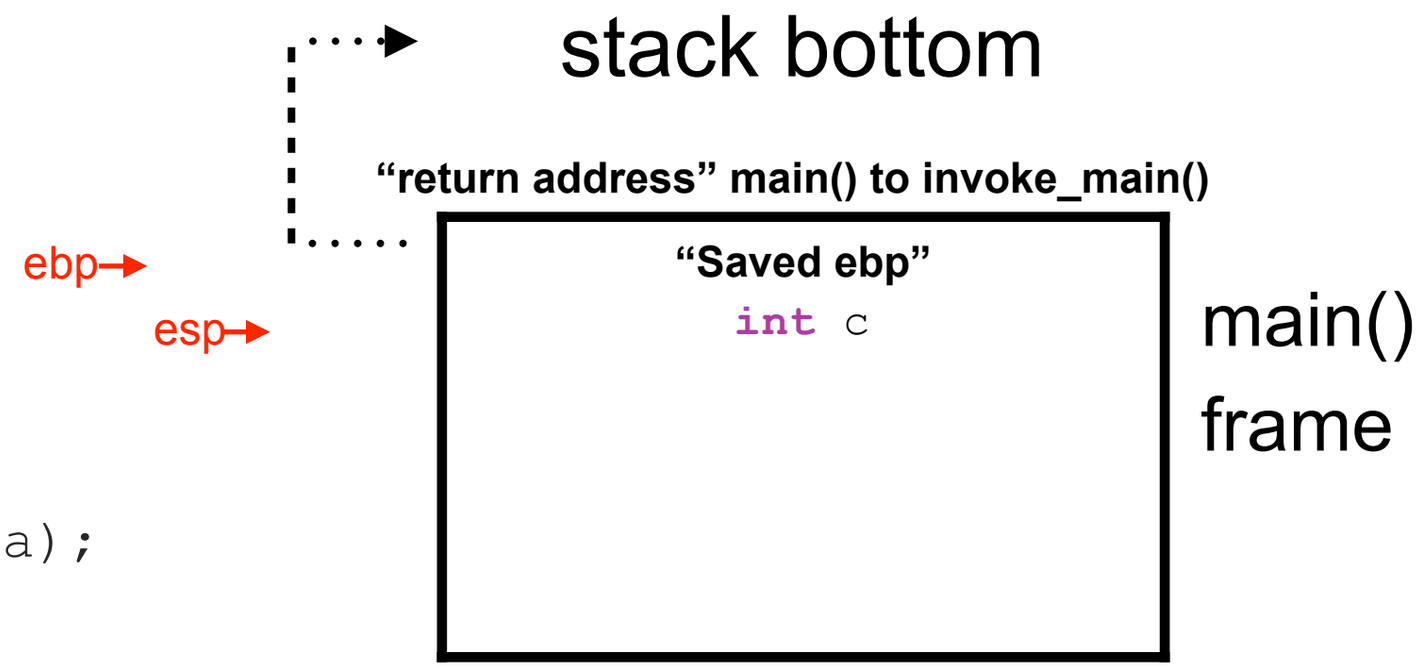
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



stack top

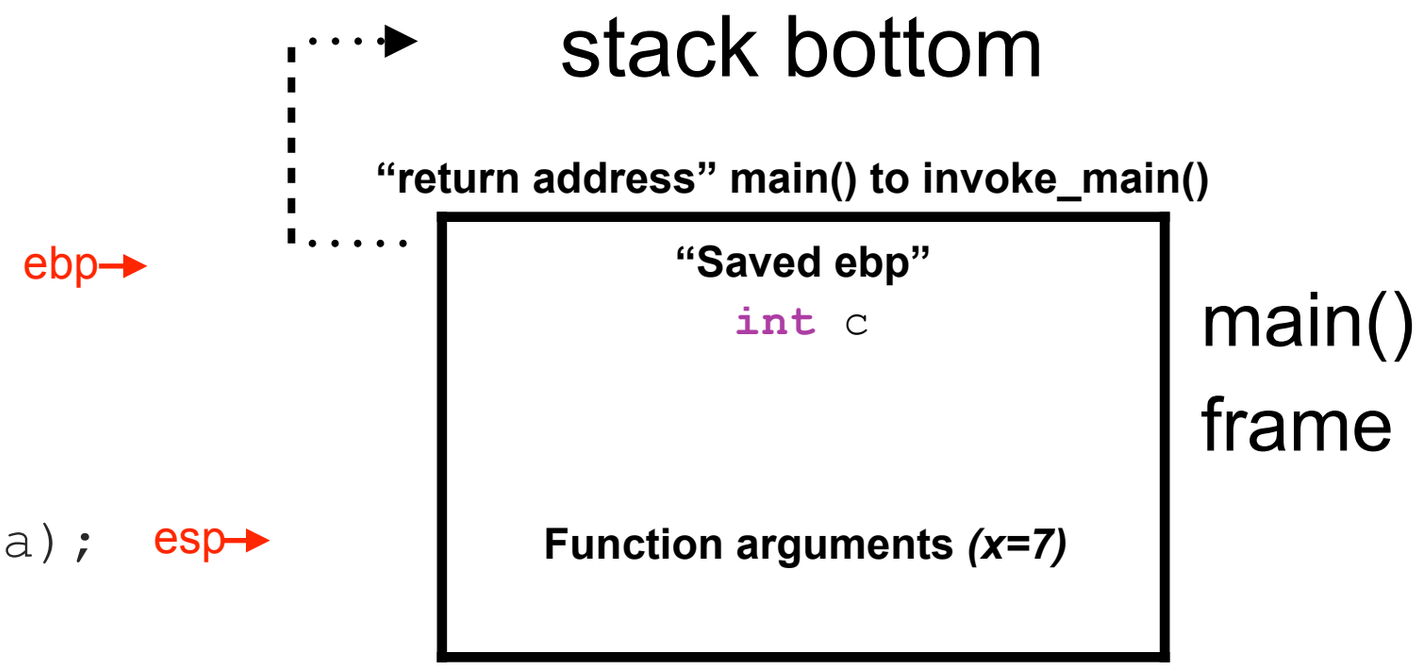
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



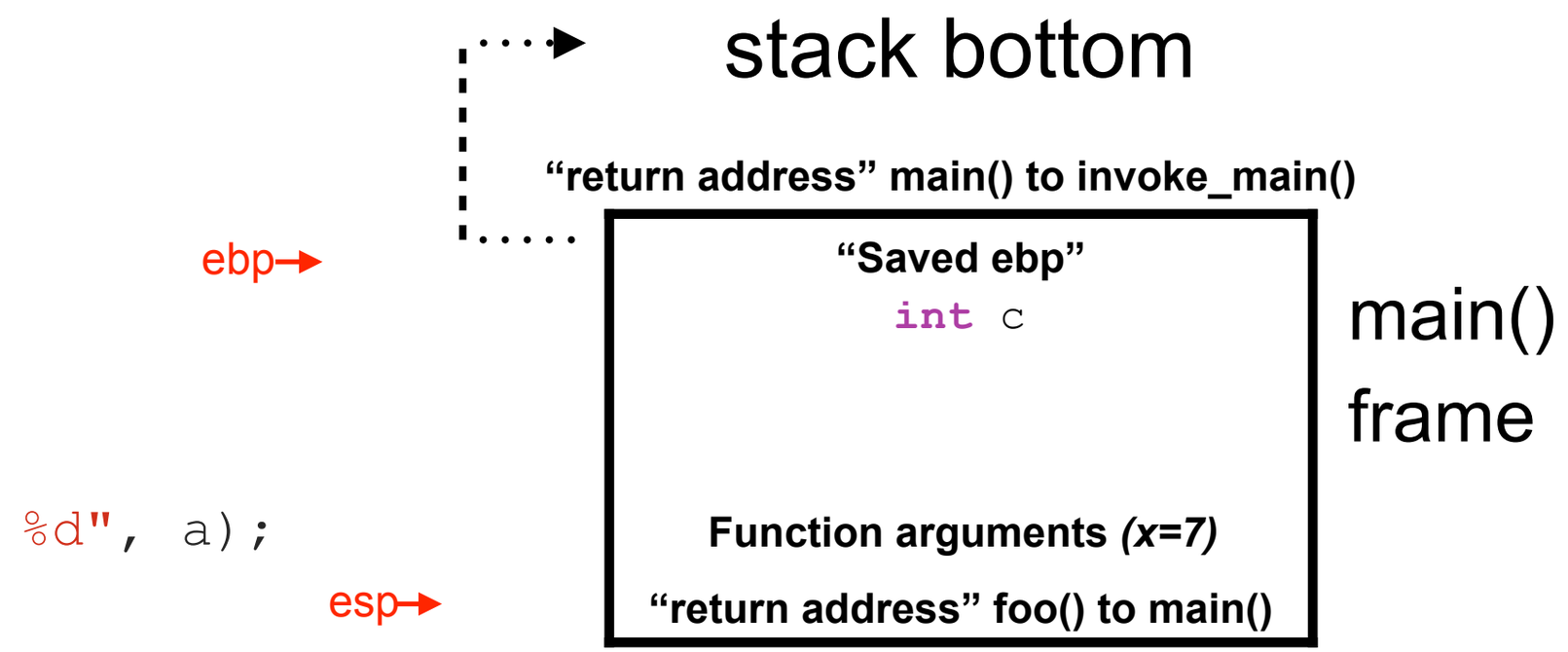
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



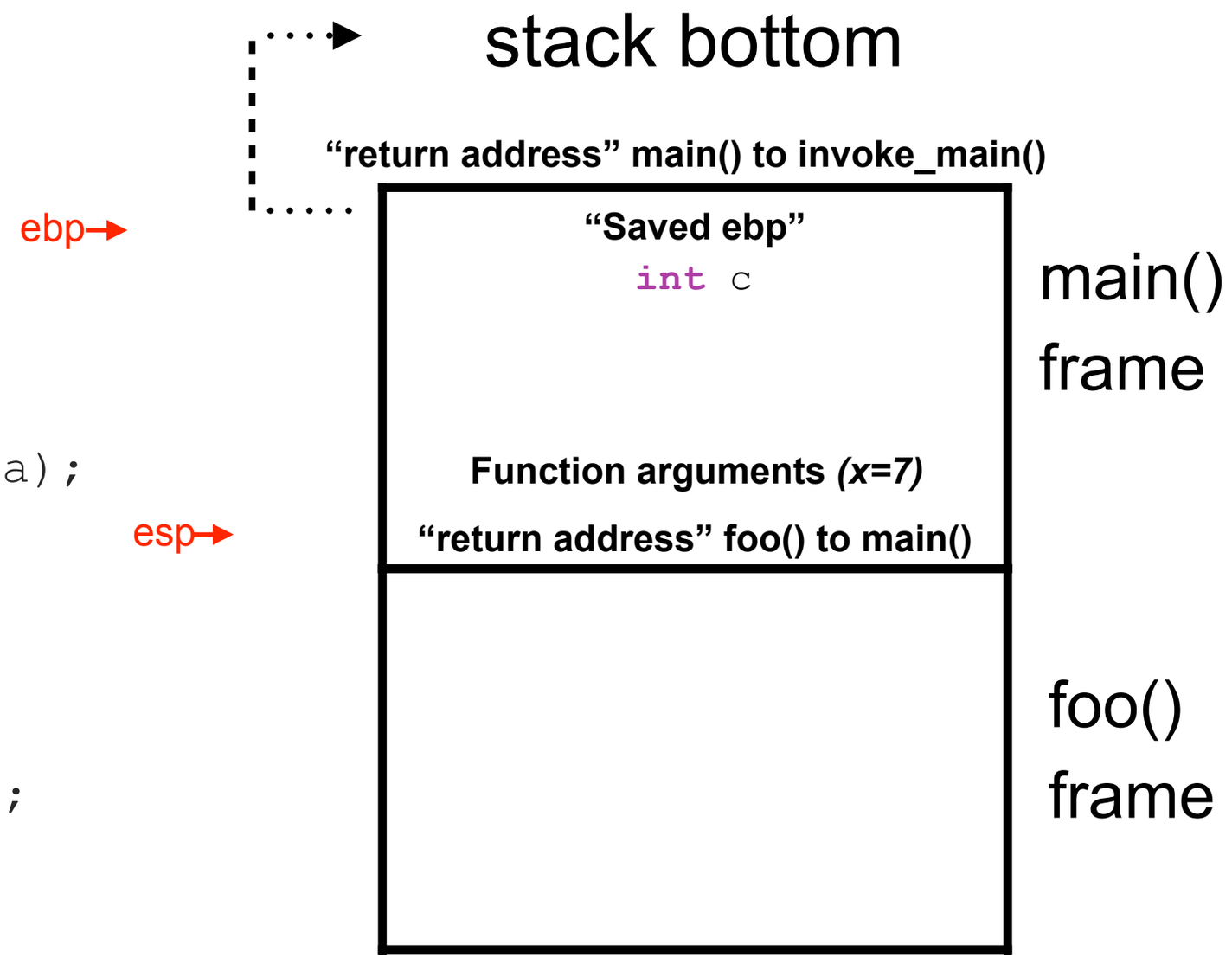
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



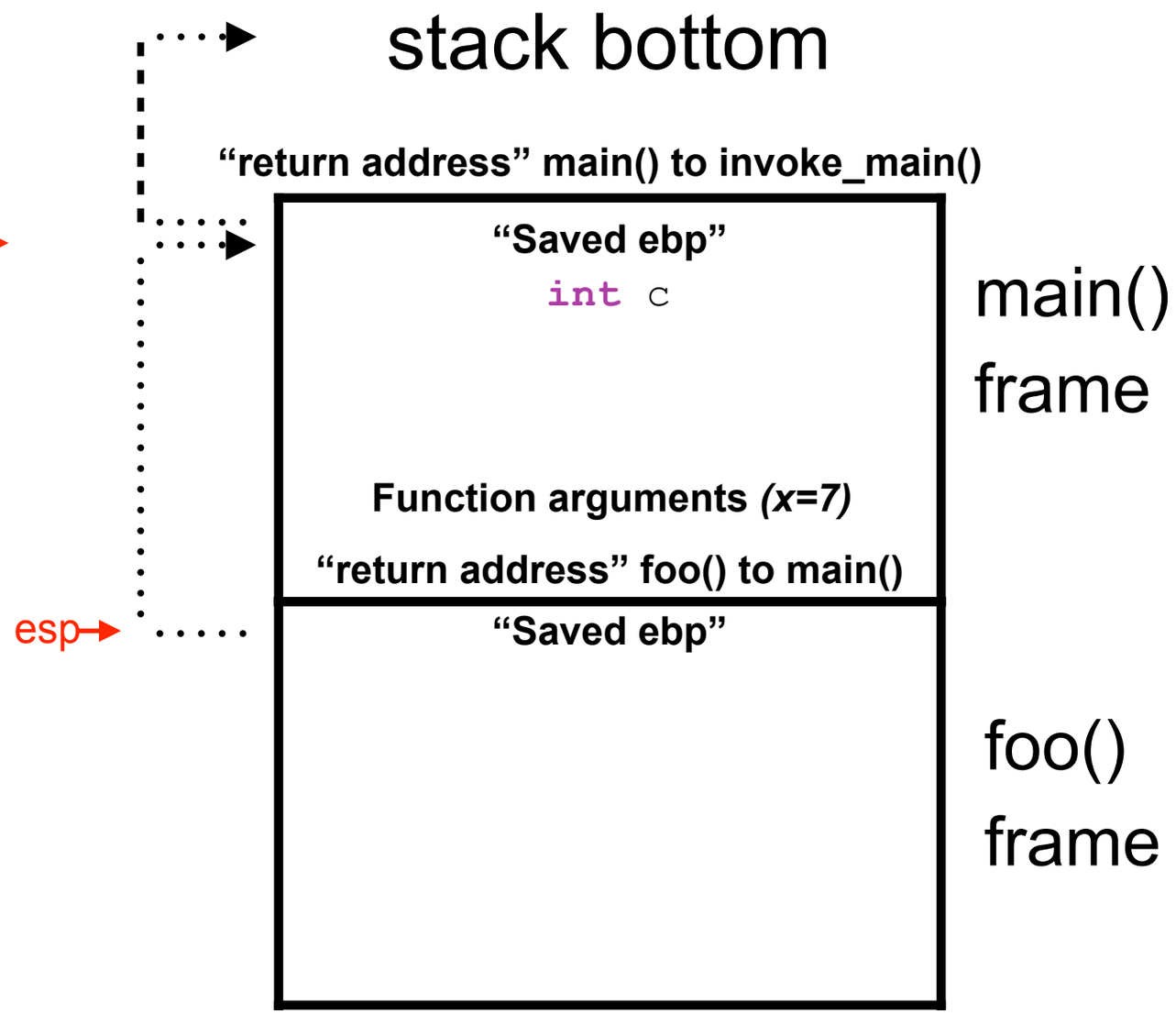
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



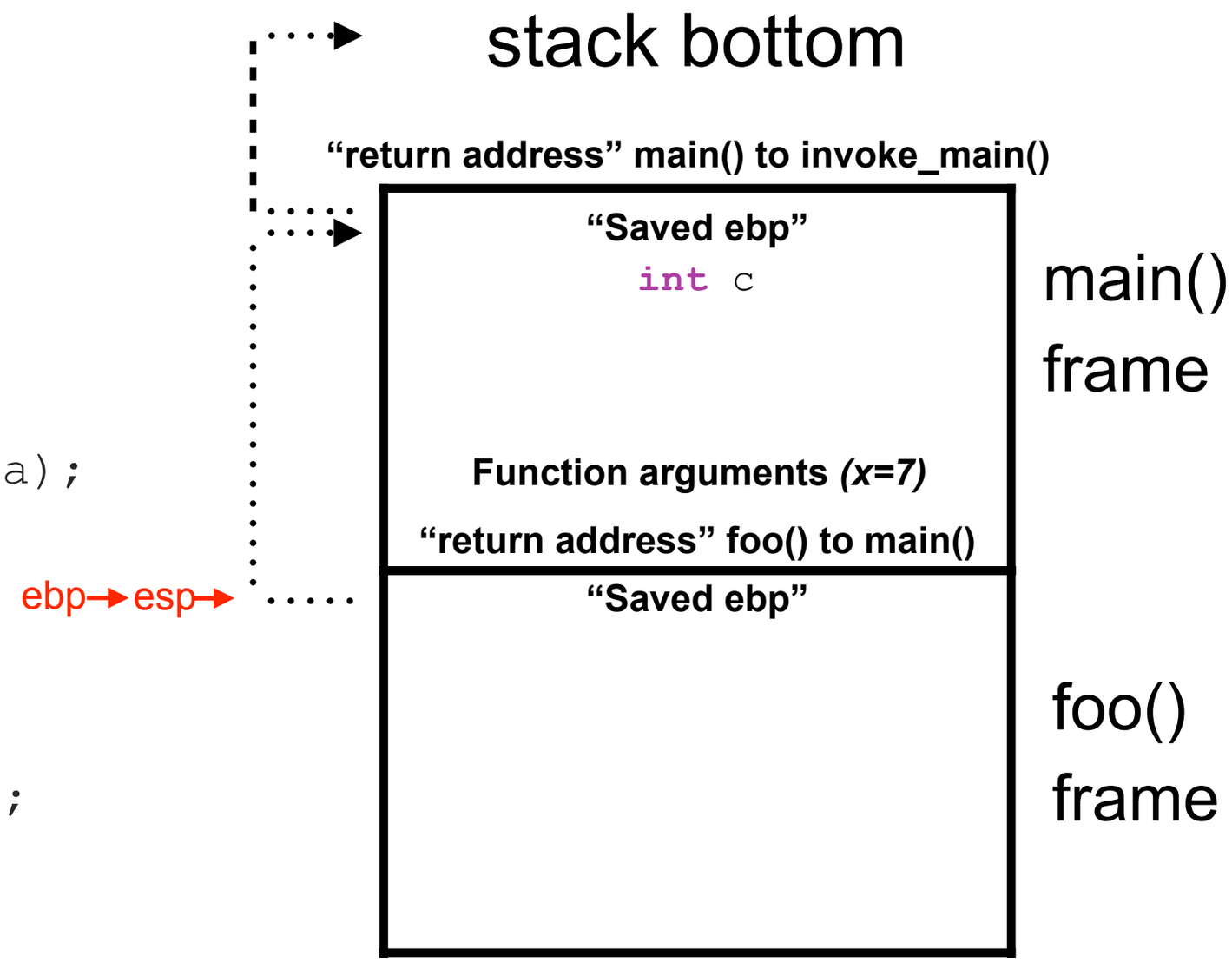
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



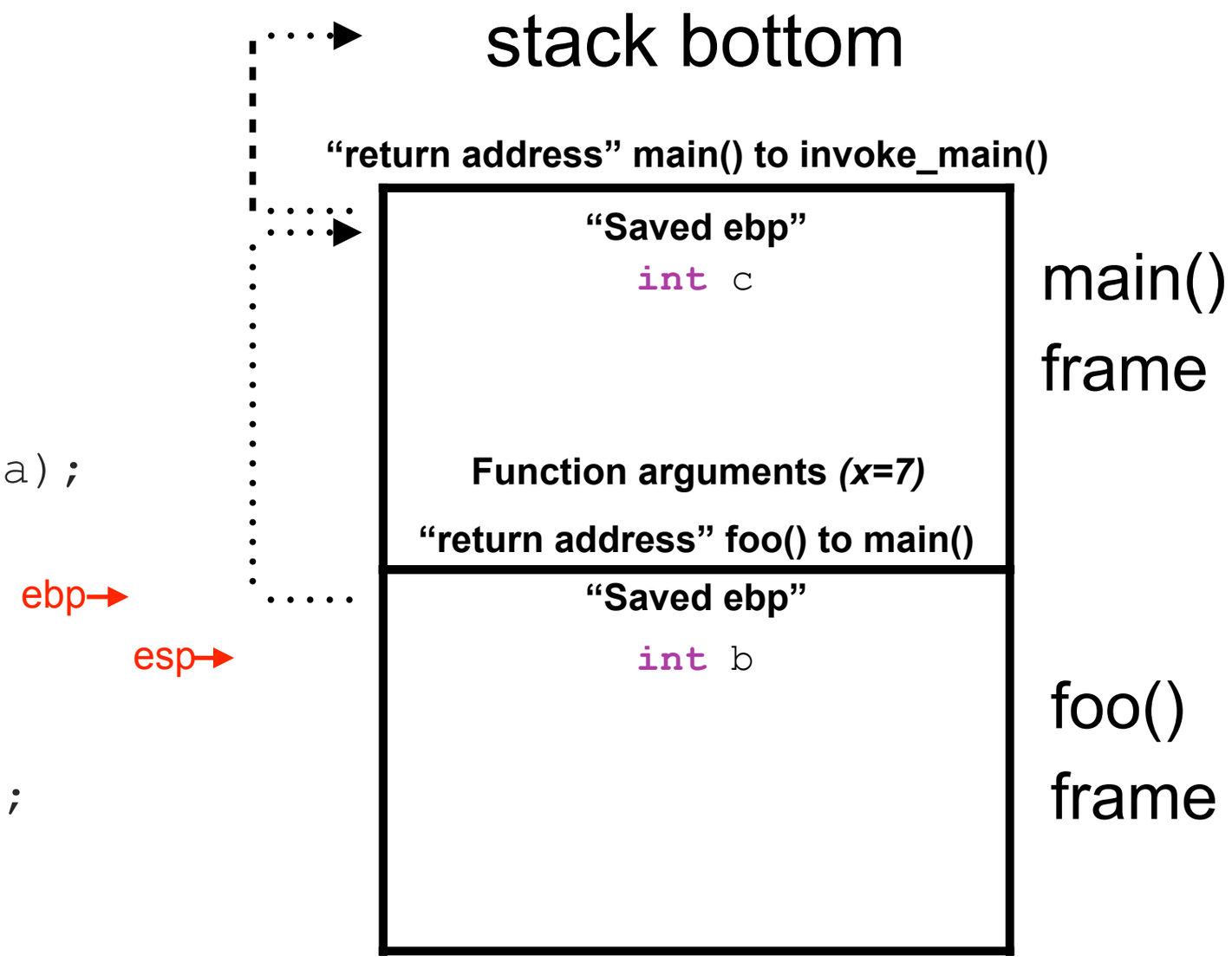
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

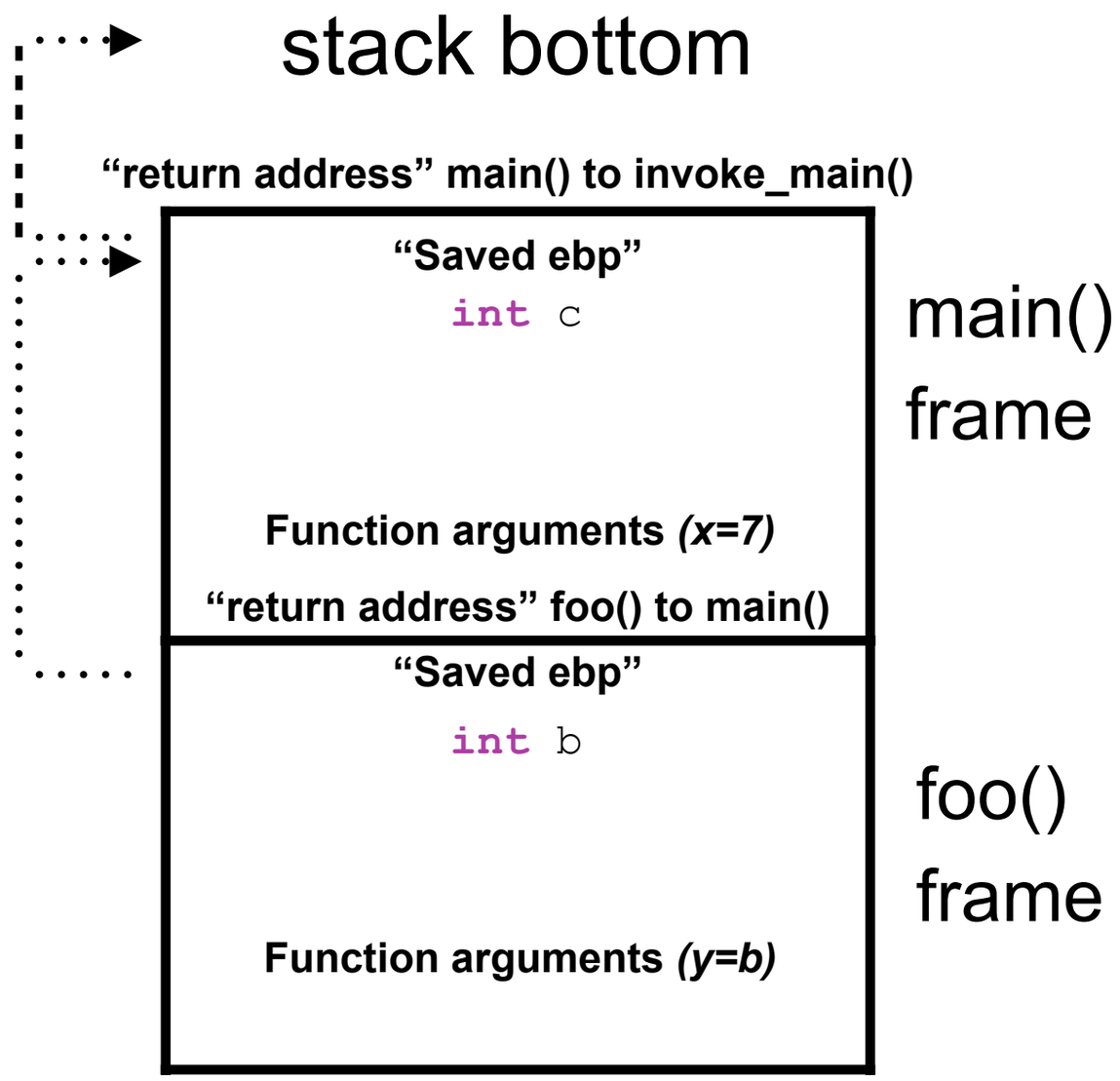
int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

ebp→

esp→



stack top

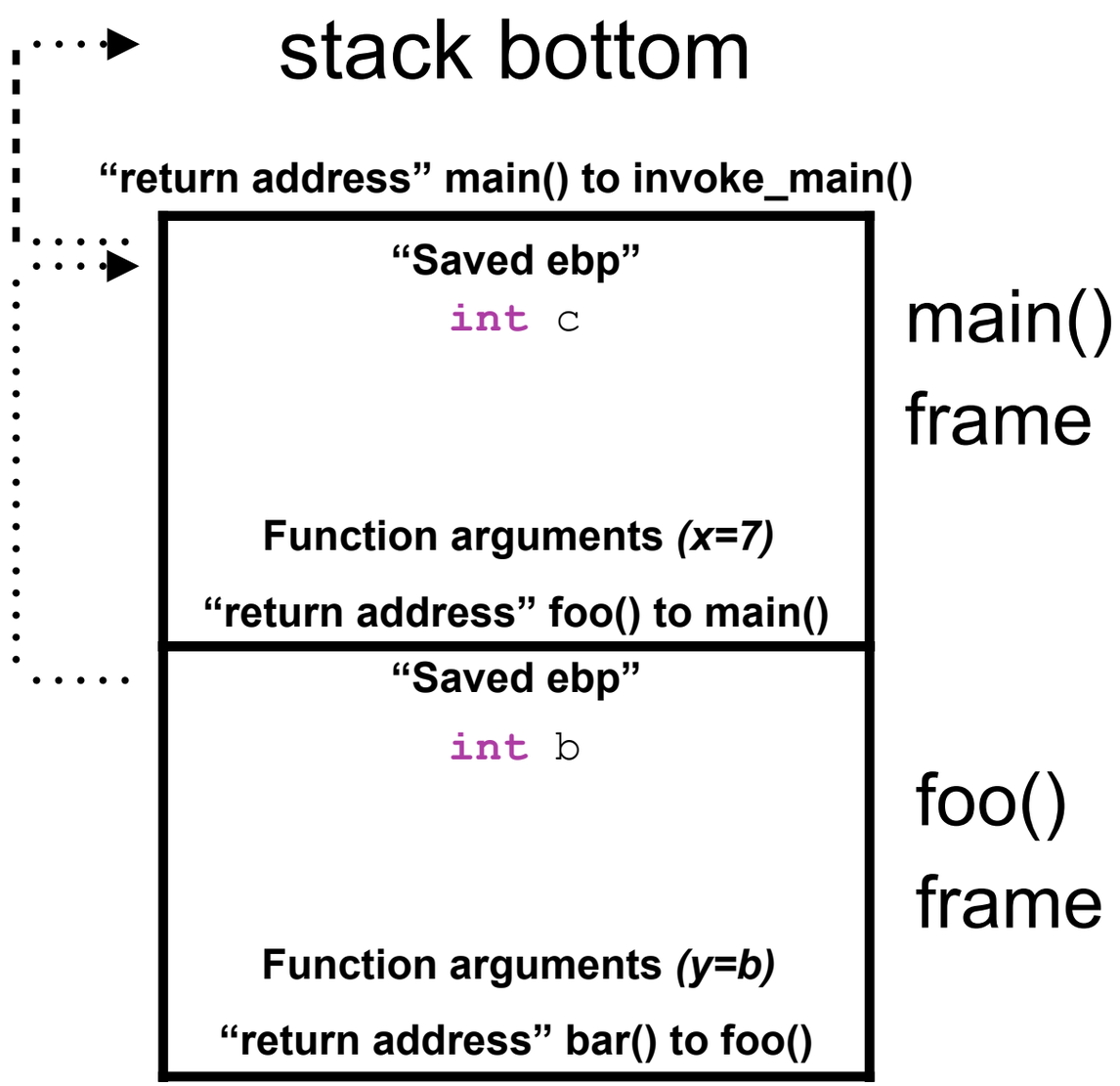
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



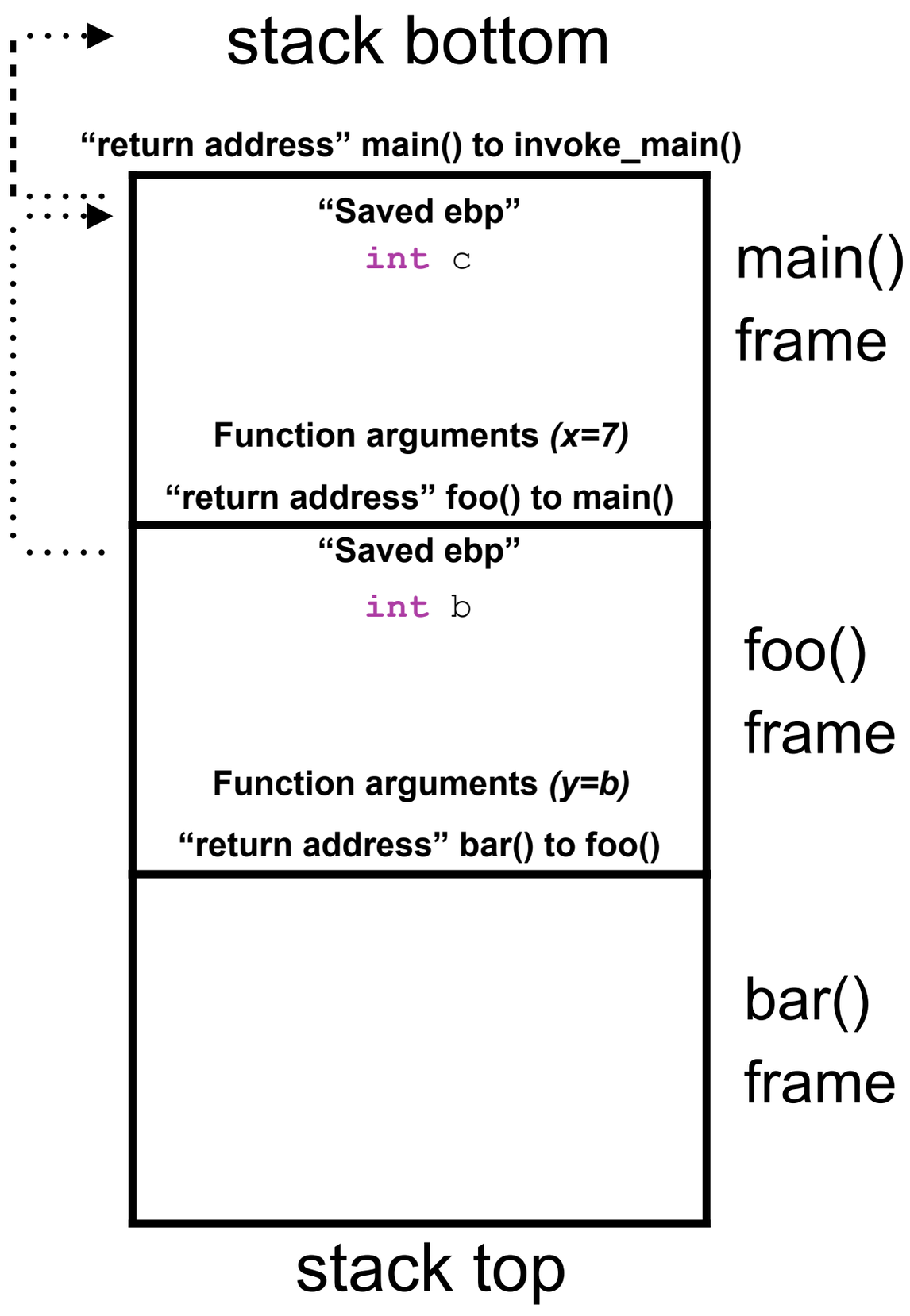
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



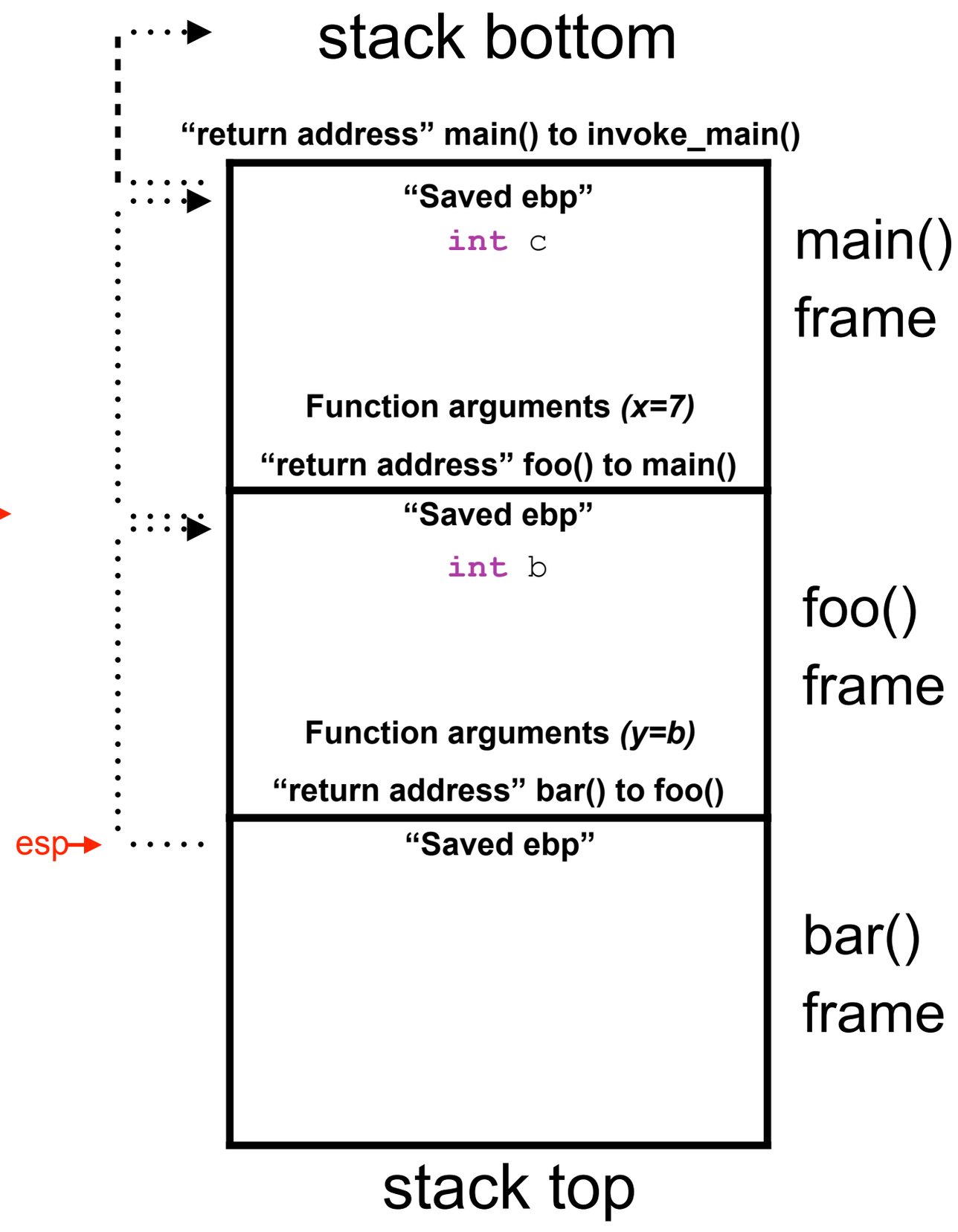
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



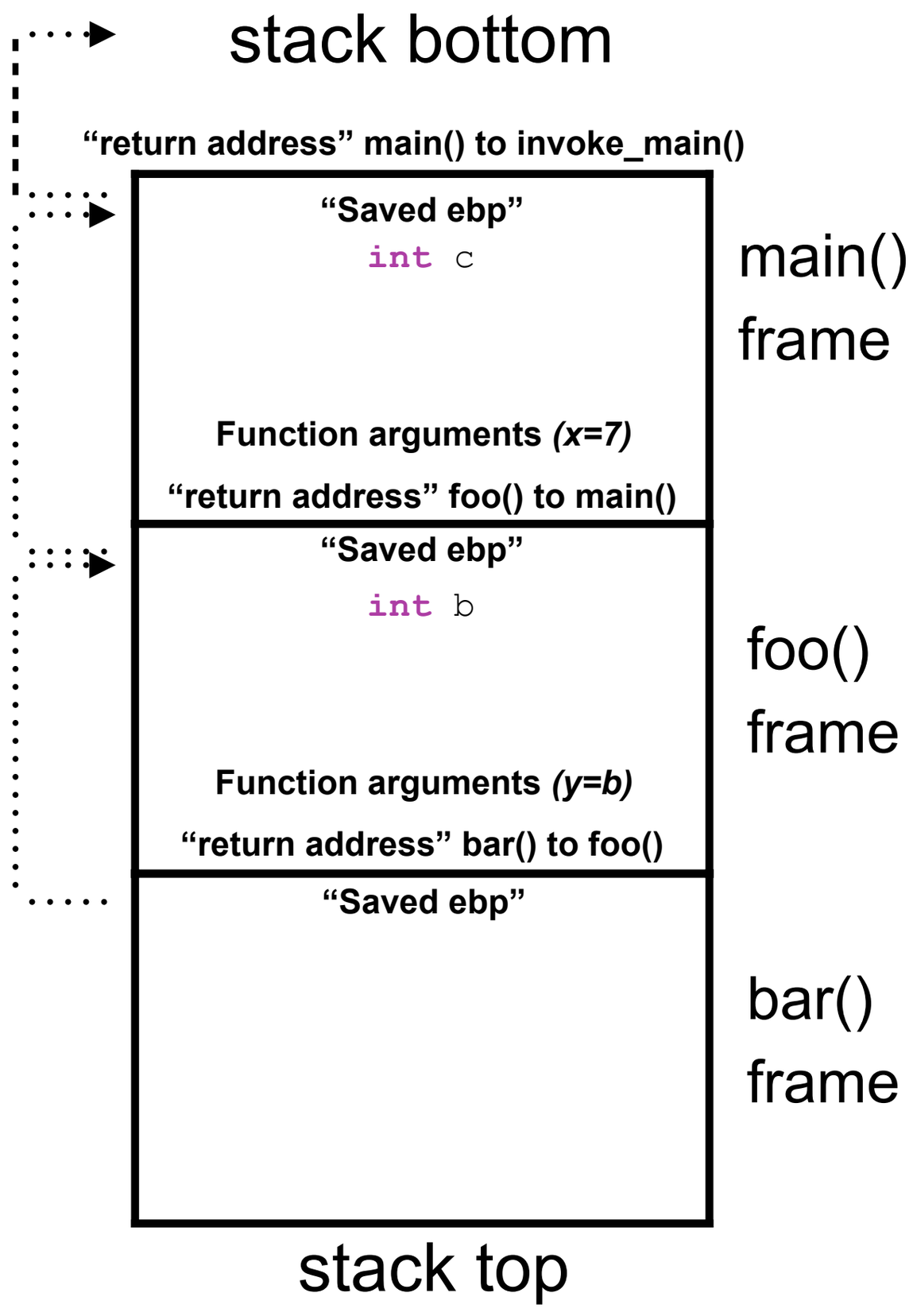
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



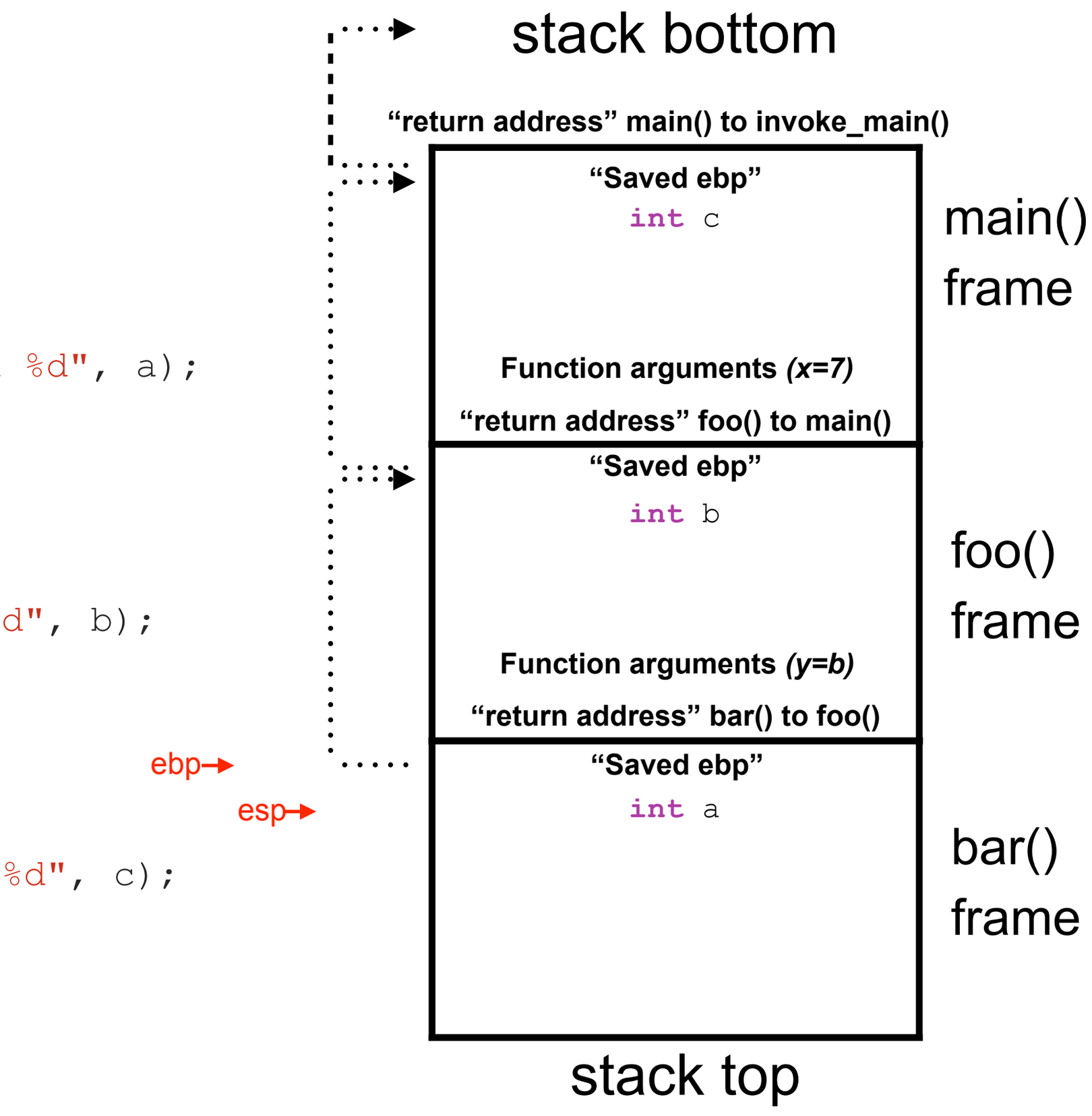
32-bit Simple Stack Diagram 4

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



32-bit Maximum Stack Diagram

stack bottom

stack top

32-bit Maximum Stack Diagram

ebp→

stack bottom

stack top

32-bit Maximum Stack Diagram

ebp→

stack bottom

“return address” from main() to whoever called main()

stack top

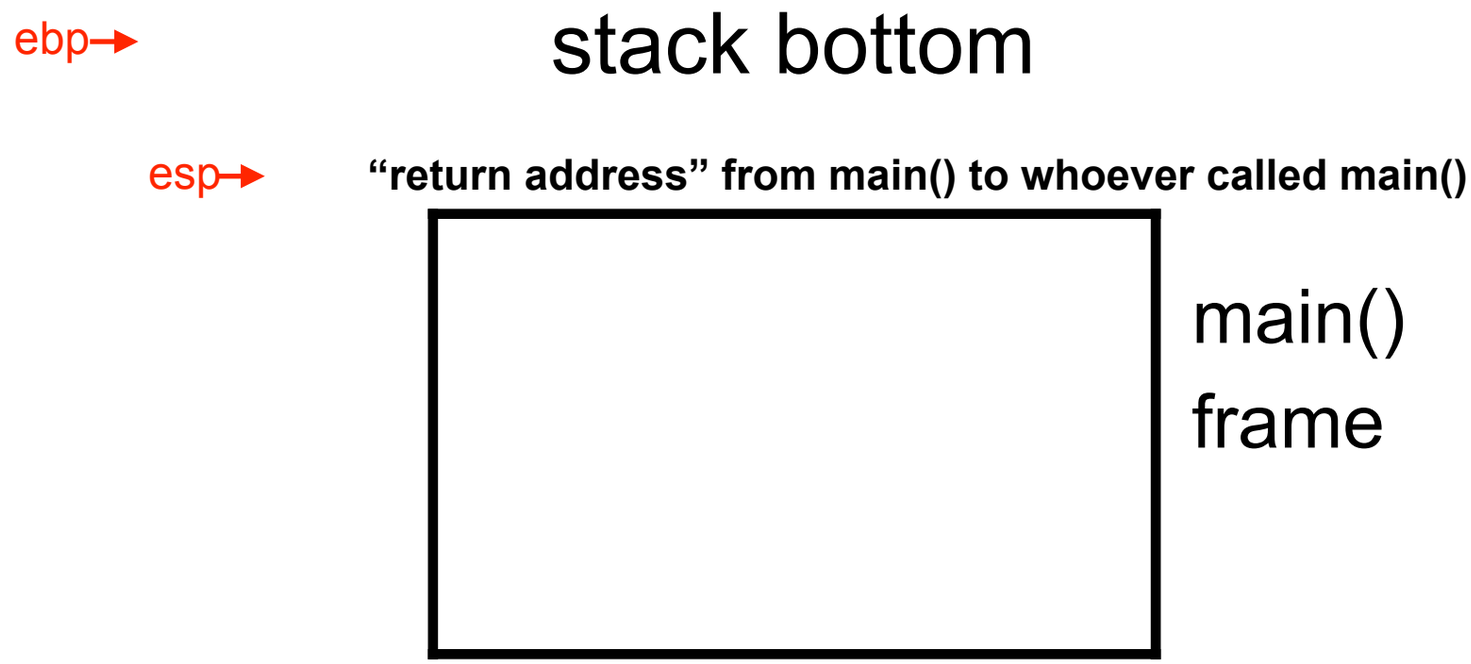
32-bit Maximum Stack Diagram

ebp→ stack bottom

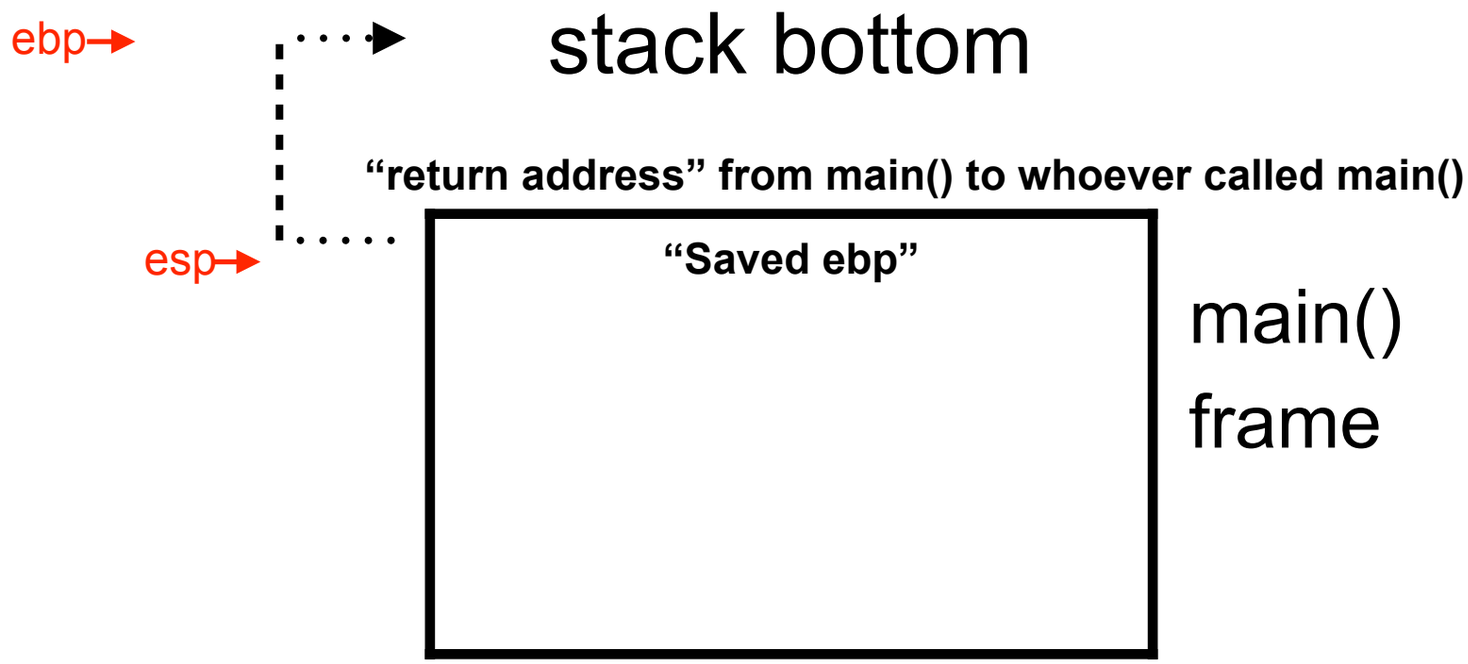
esp→ **“return address” from main() to whoever called main()**

stack top

32-bit Maximum Stack Diagram

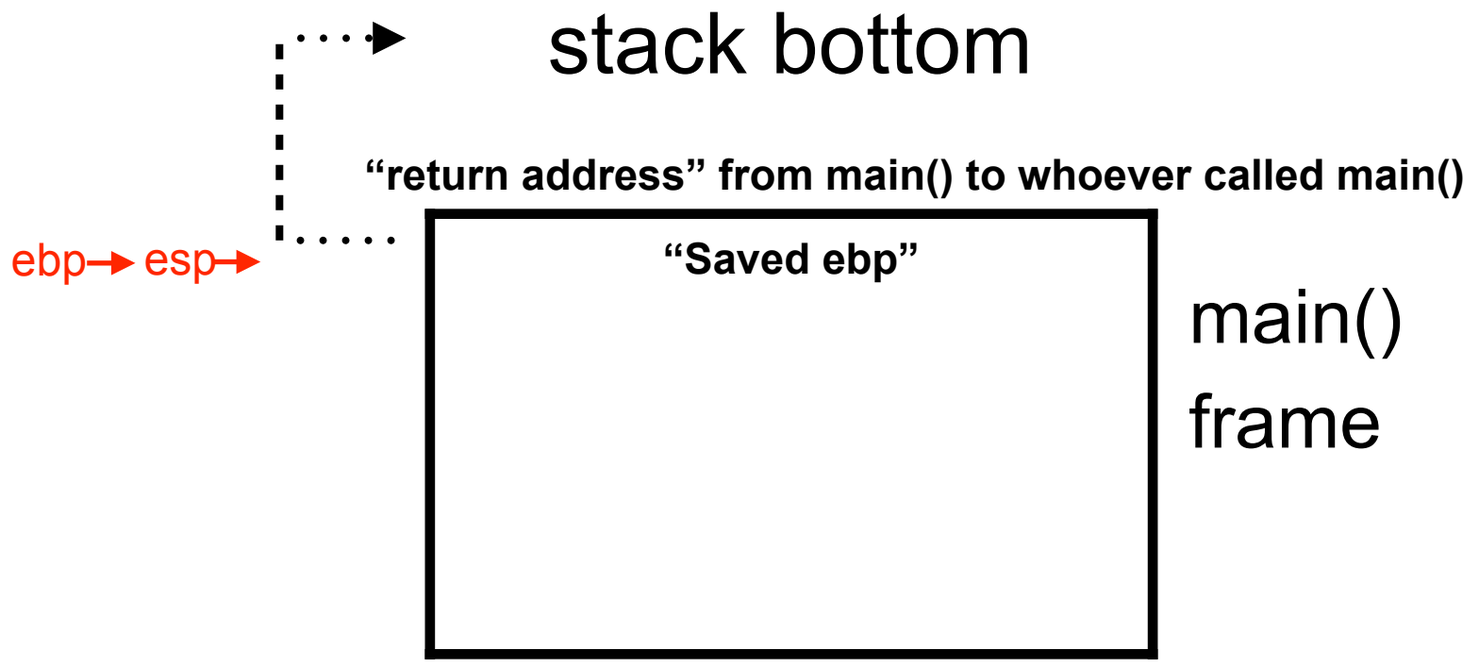


32-bit Maximum Stack Diagram



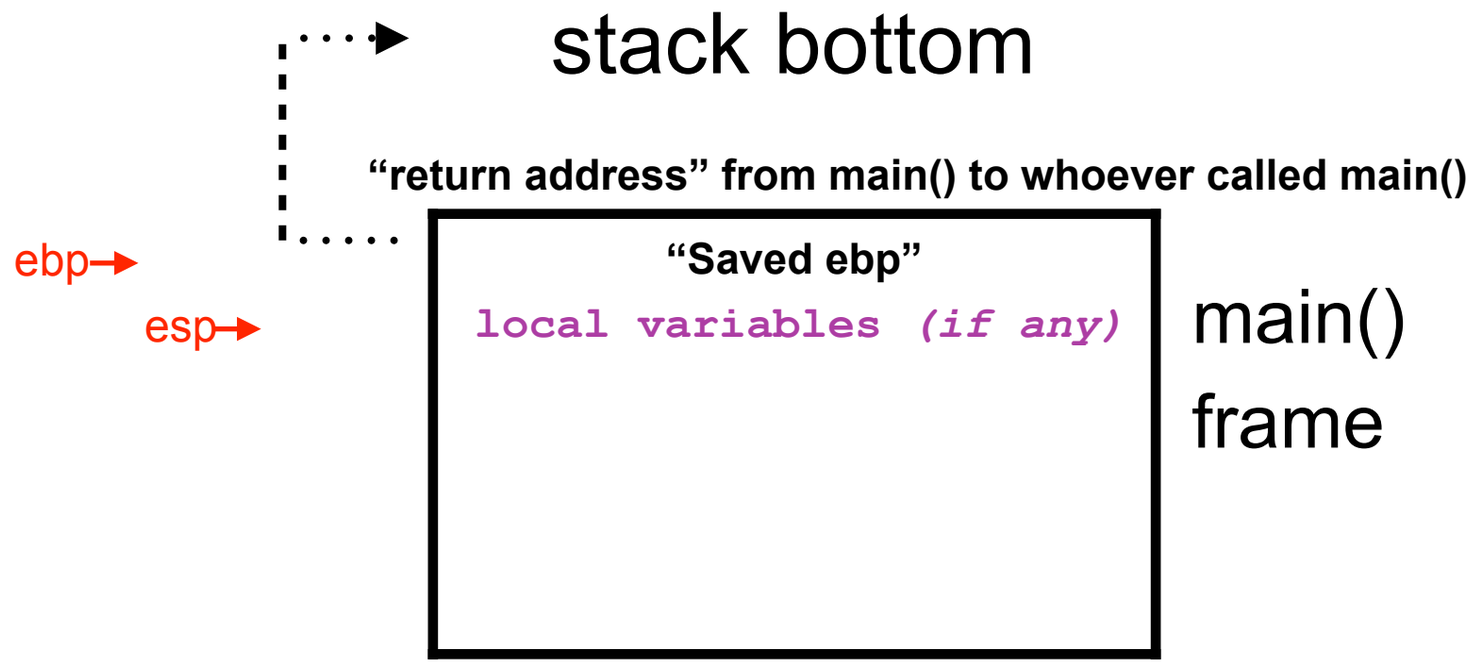
stack top

32-bit Maximum Stack Diagram



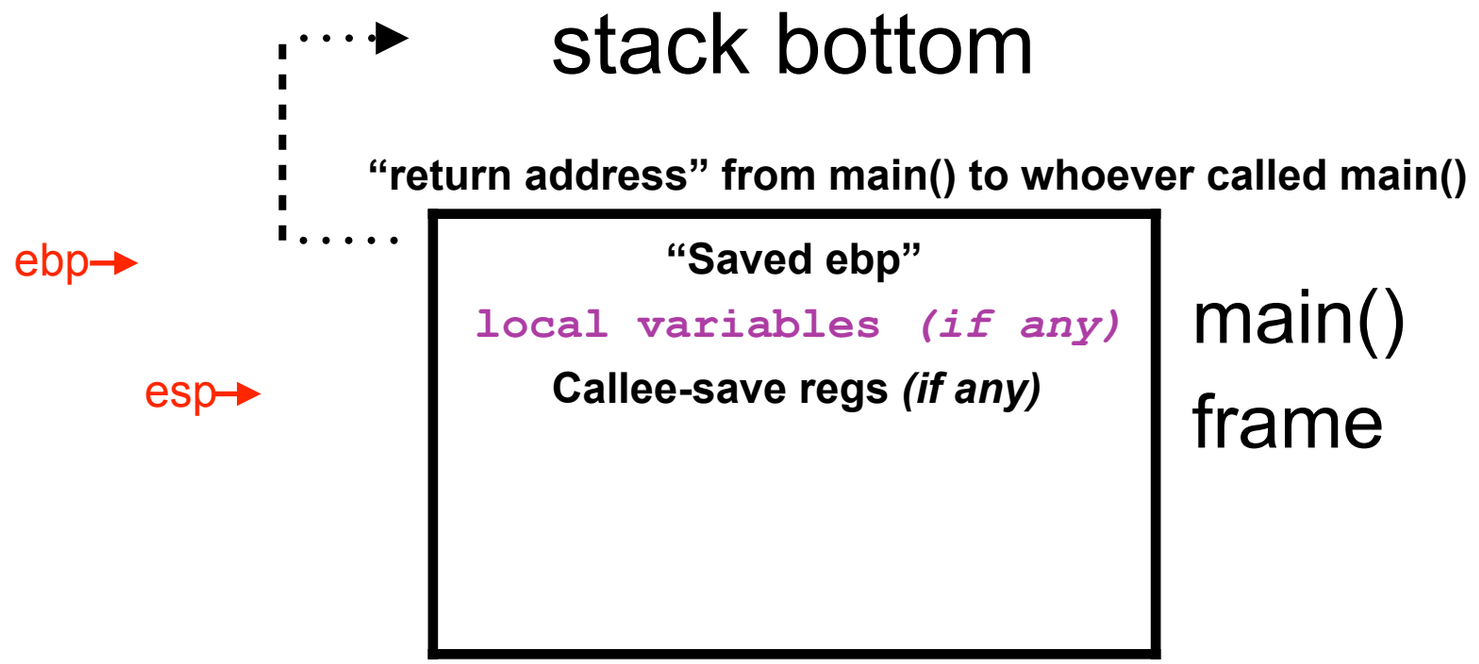
stack top

32-bit Maximum Stack Diagram



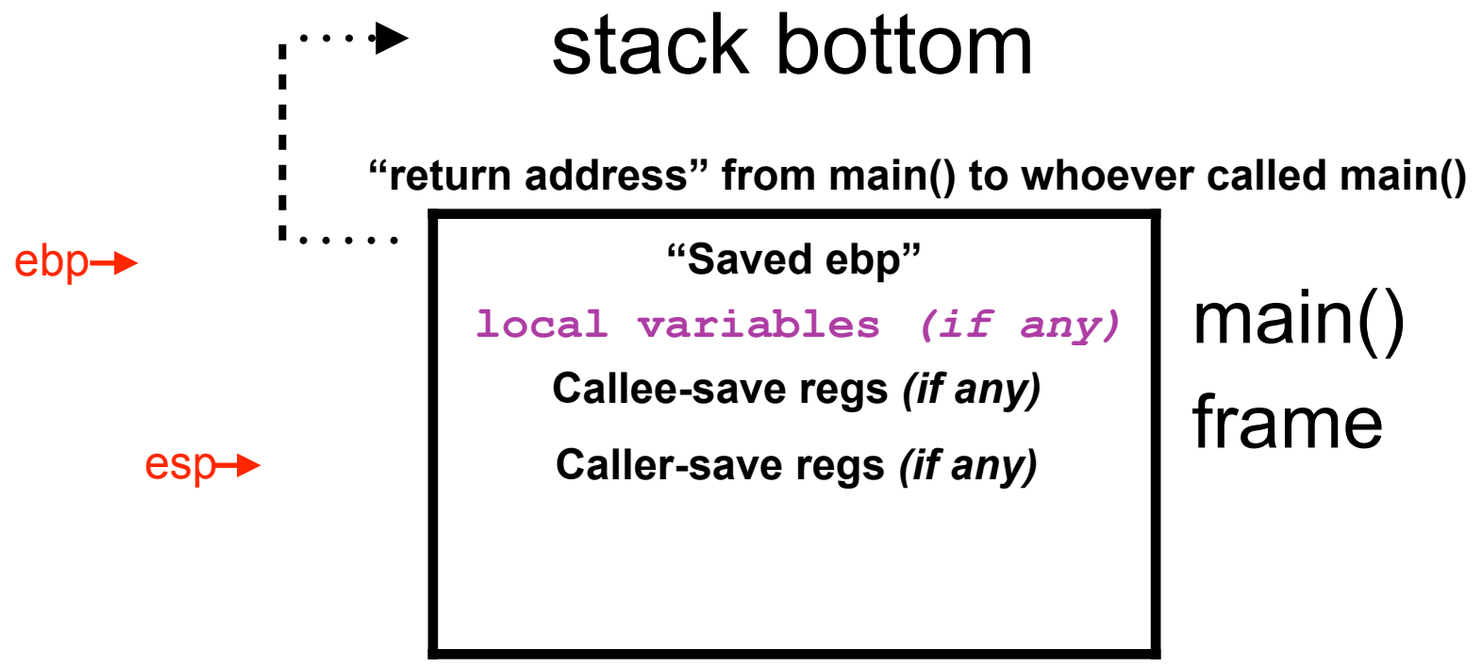
stack top

32-bit Maximum Stack Diagram



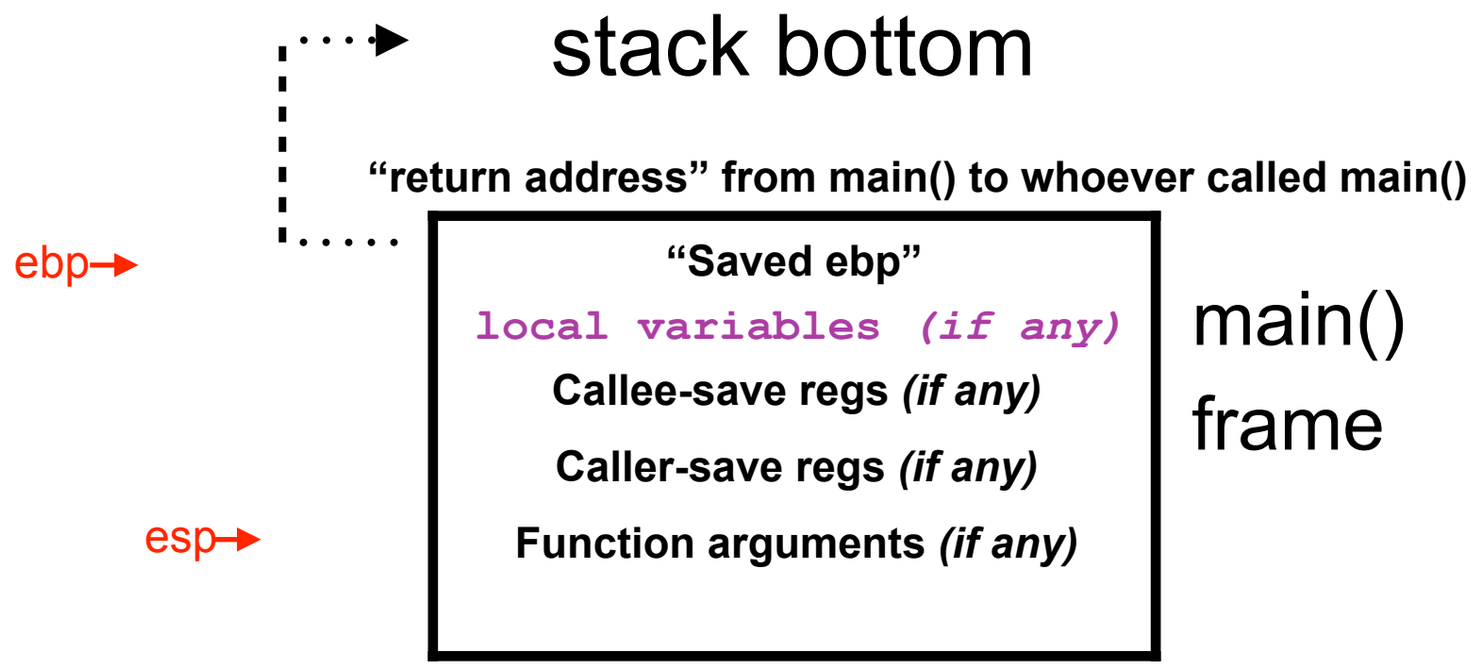
stack top

32-bit Maximum Stack Diagram



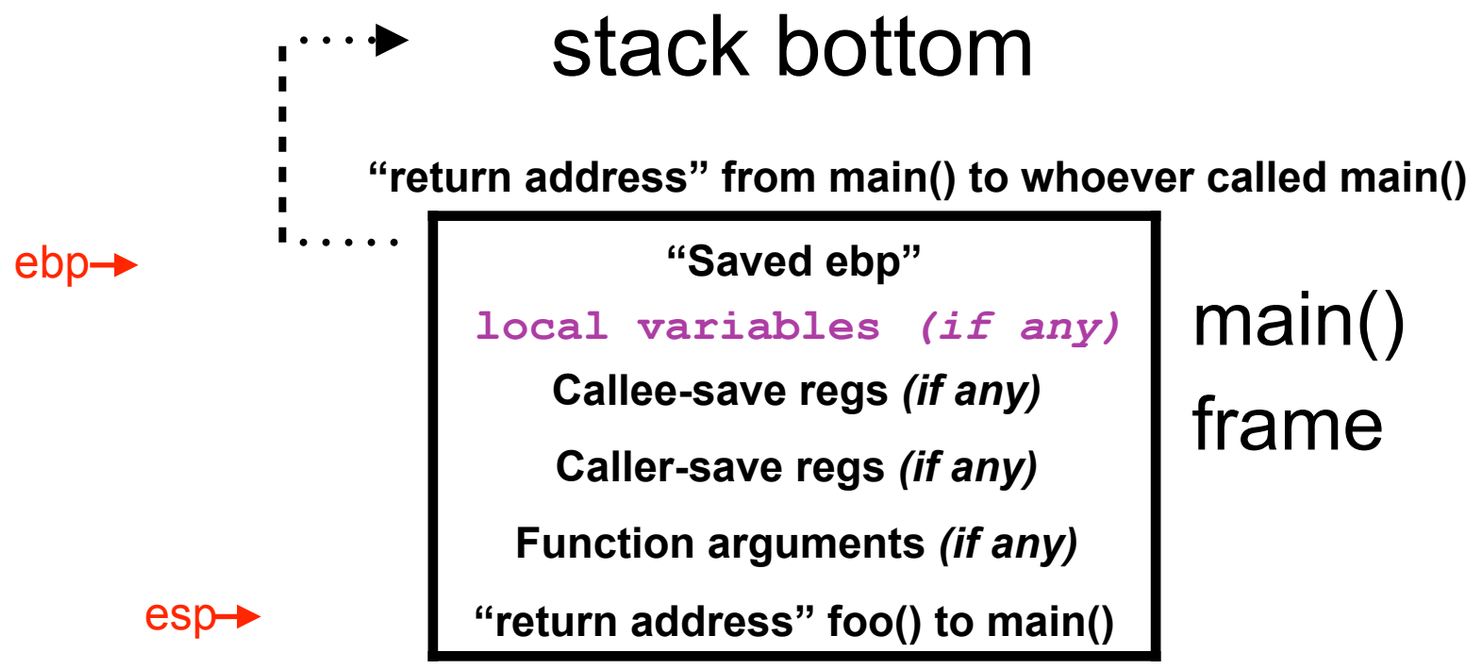
stack top

32-bit Maximum Stack Diagram



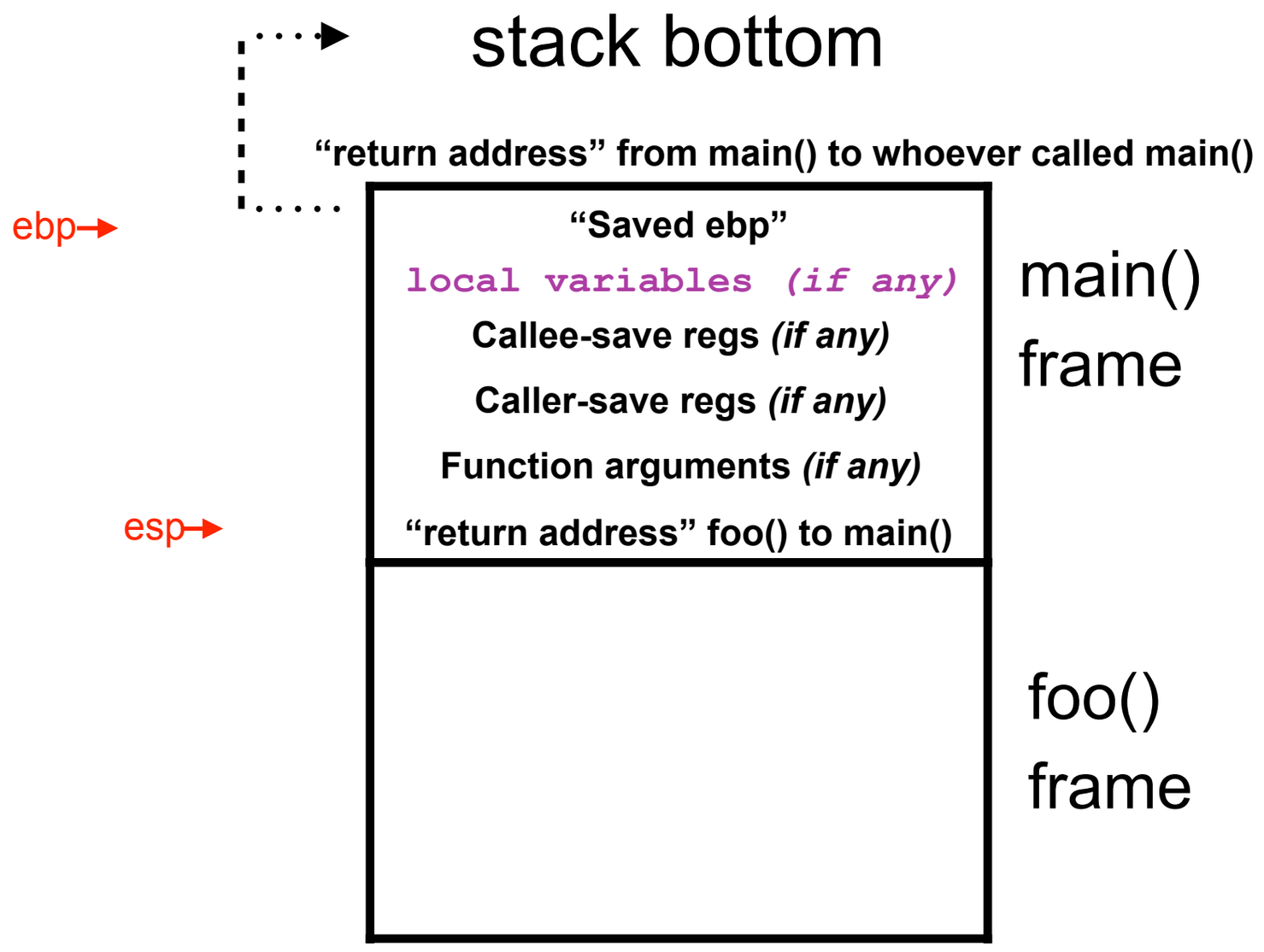
stack top

32-bit Maximum Stack Diagram

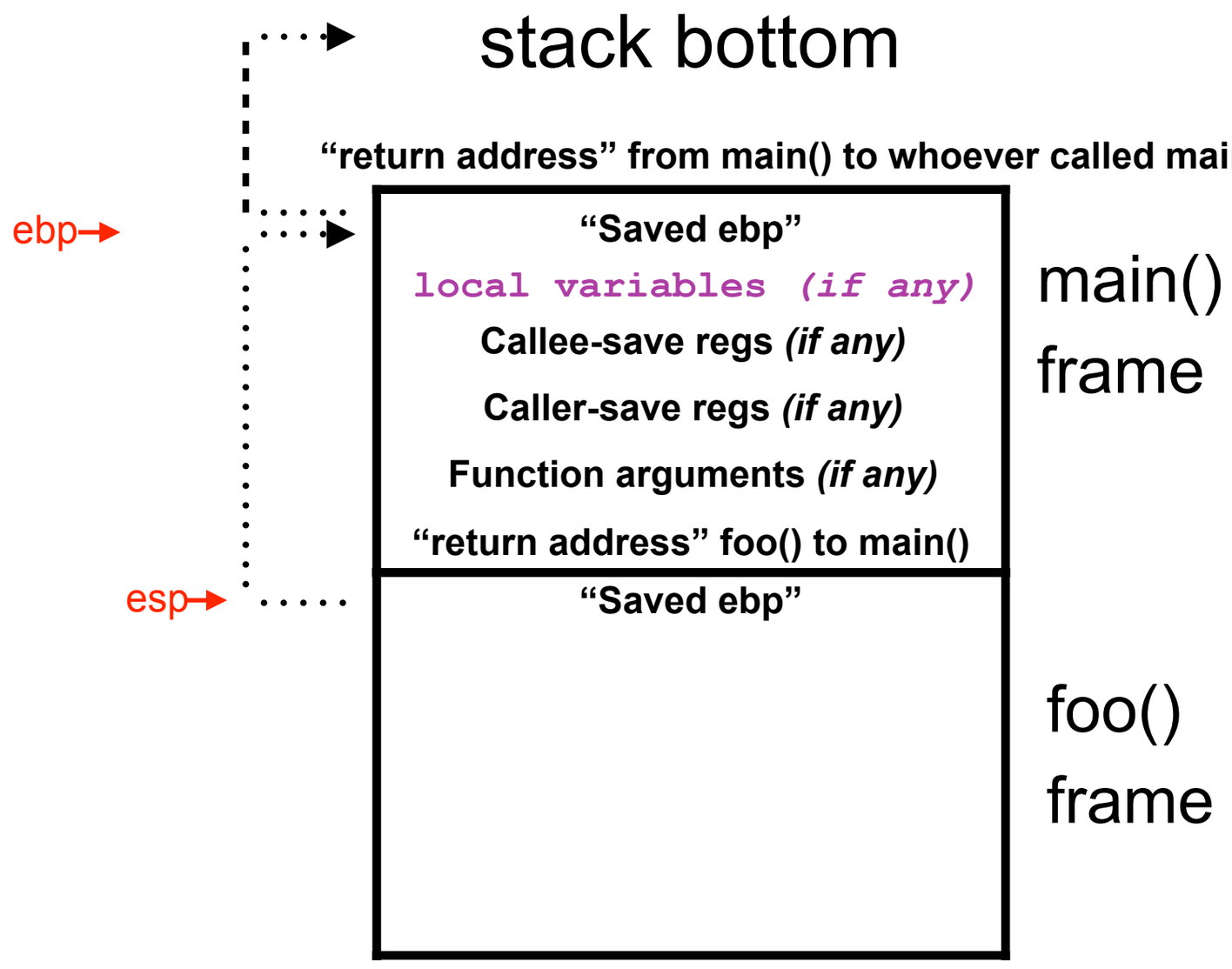


stack top

32-bit Maximum Stack Diagram

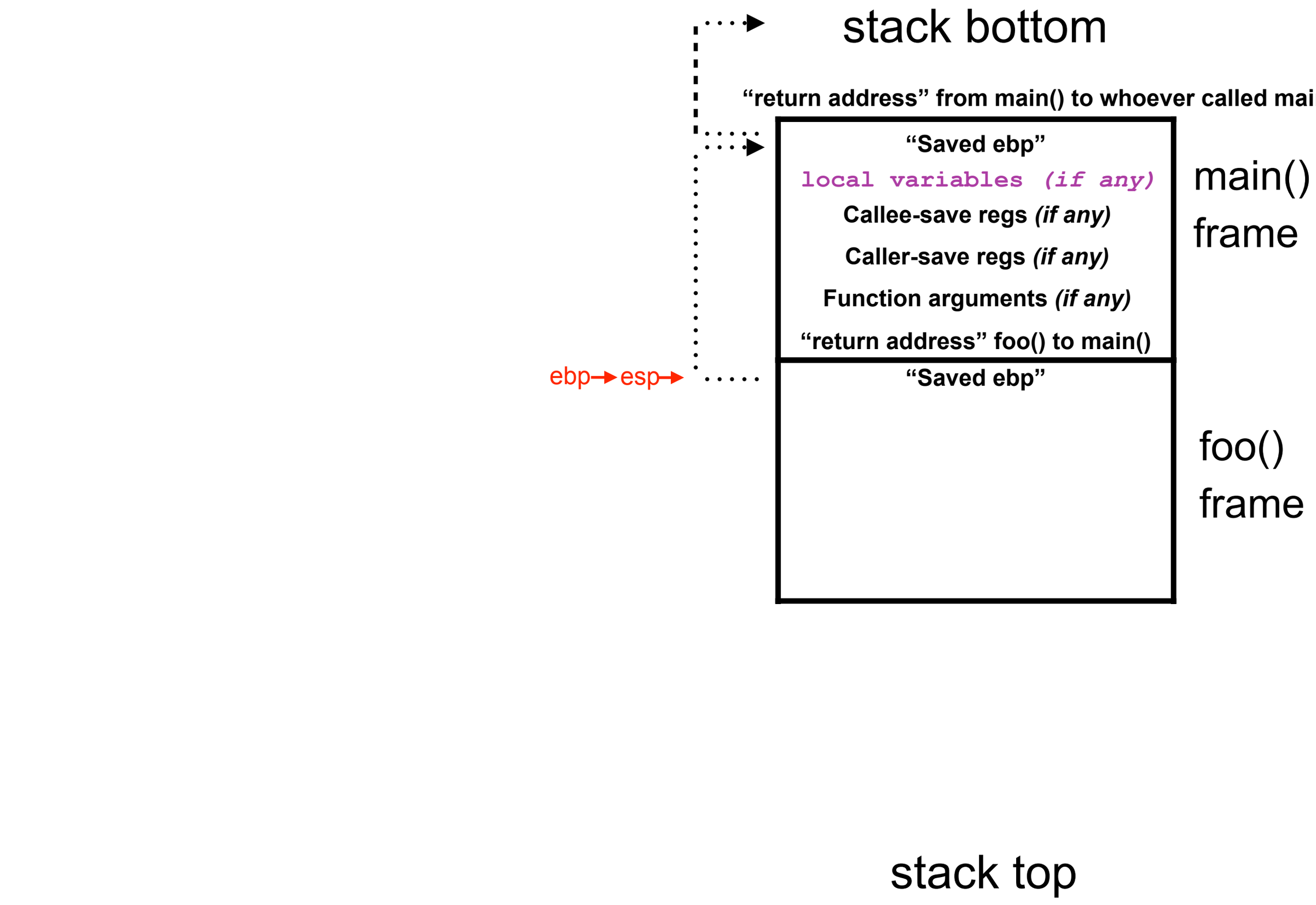


32-bit Maximum Stack Diagram

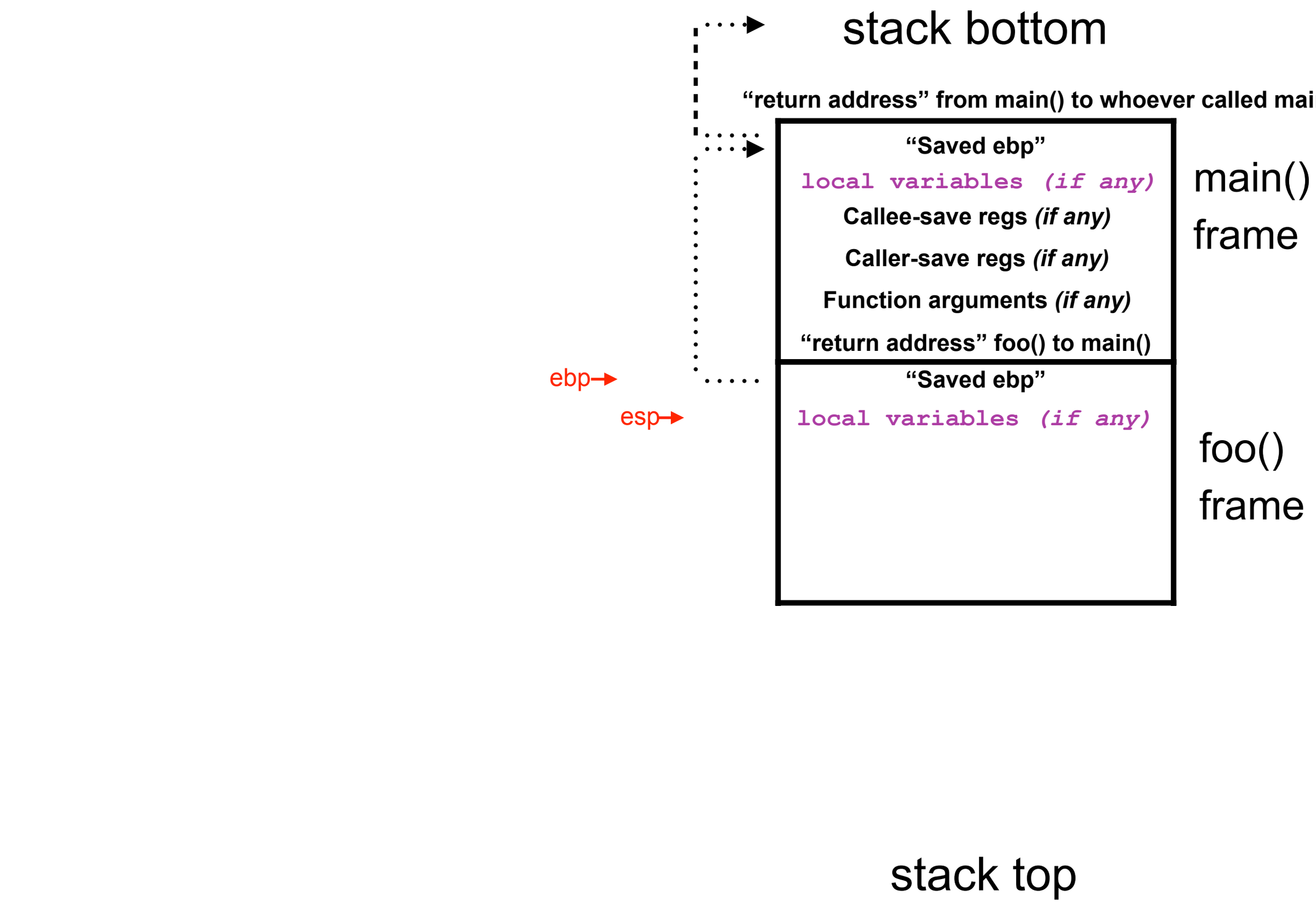


stack top

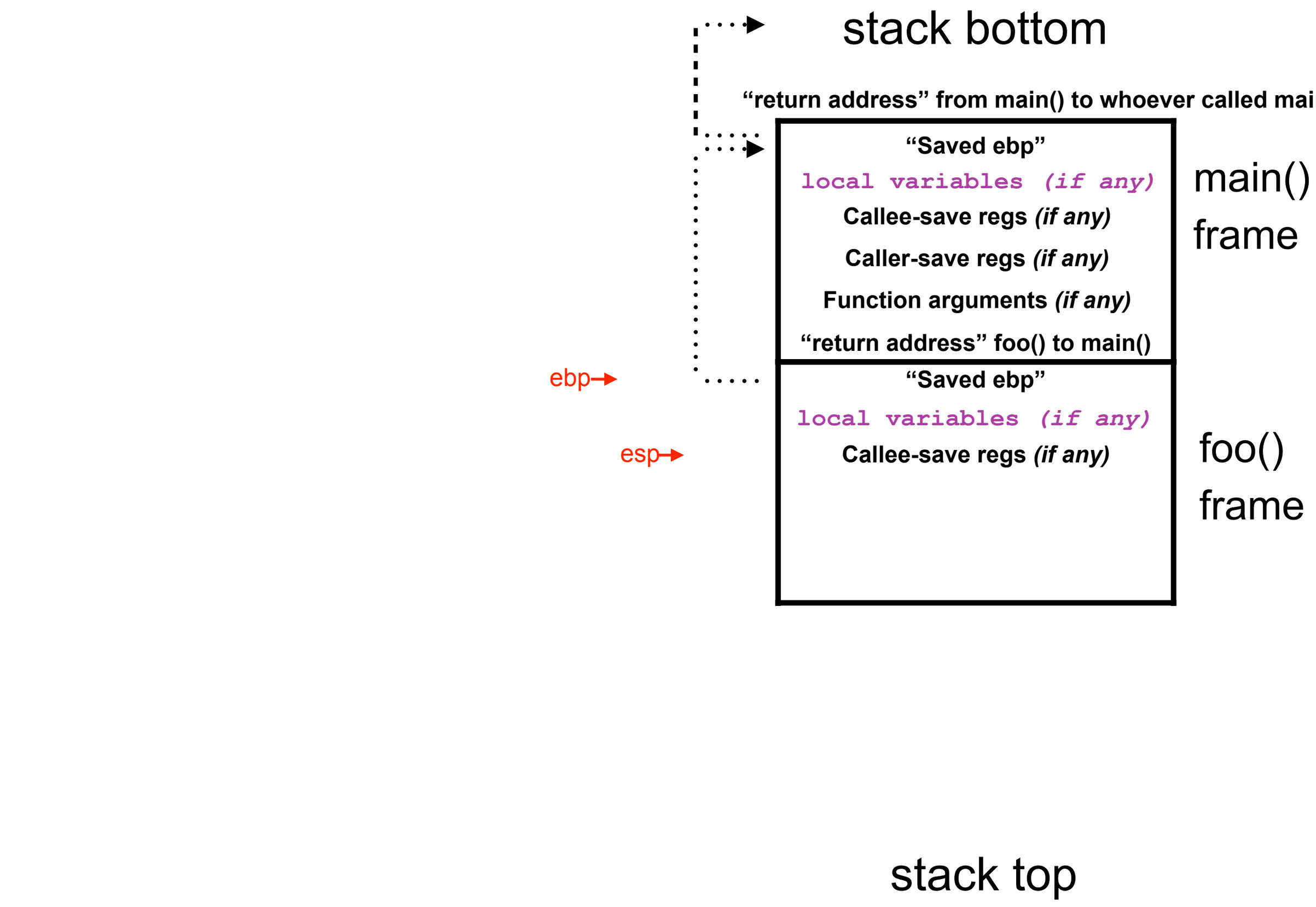
32-bit Maximum Stack Diagram



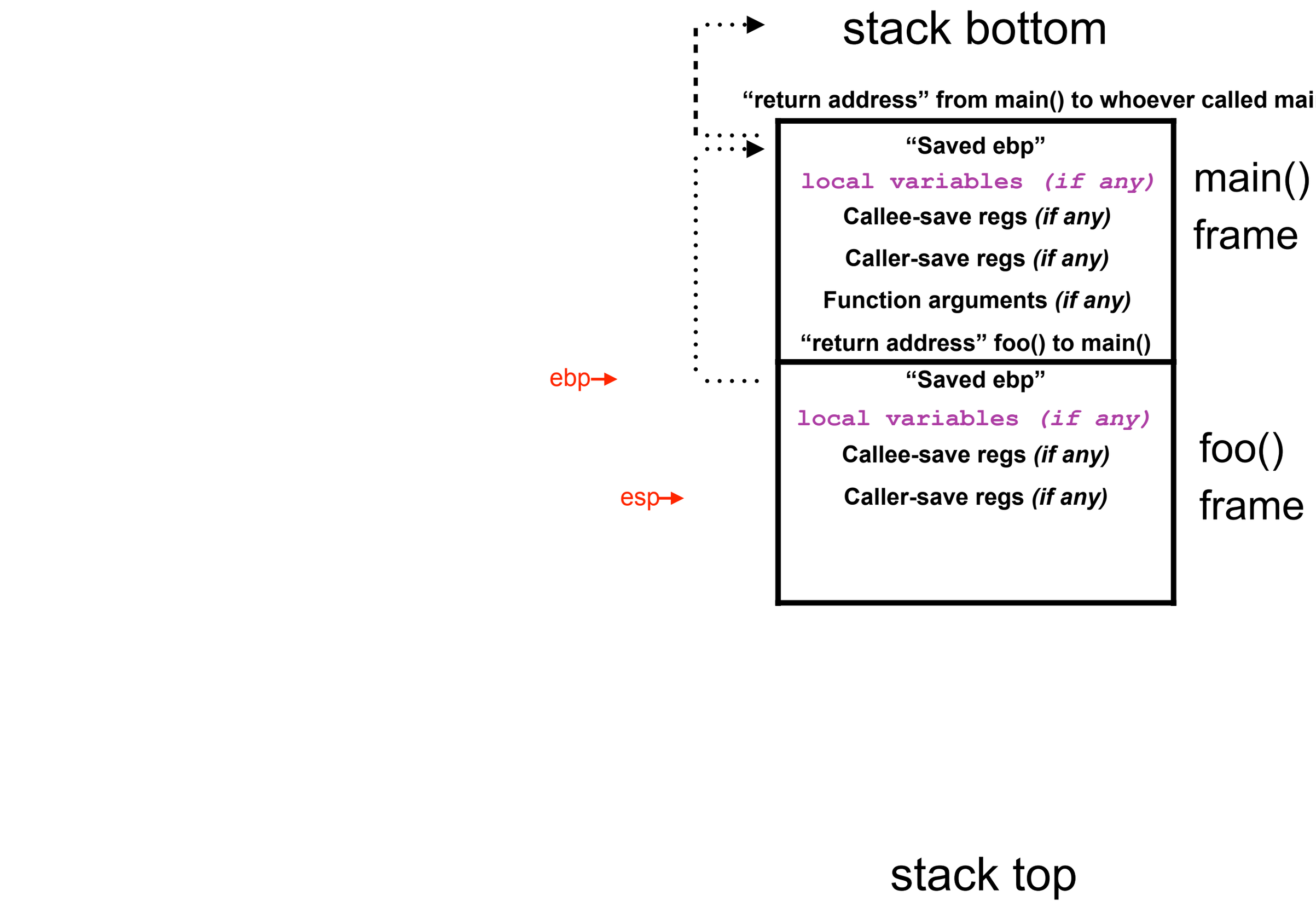
32-bit Maximum Stack Diagram



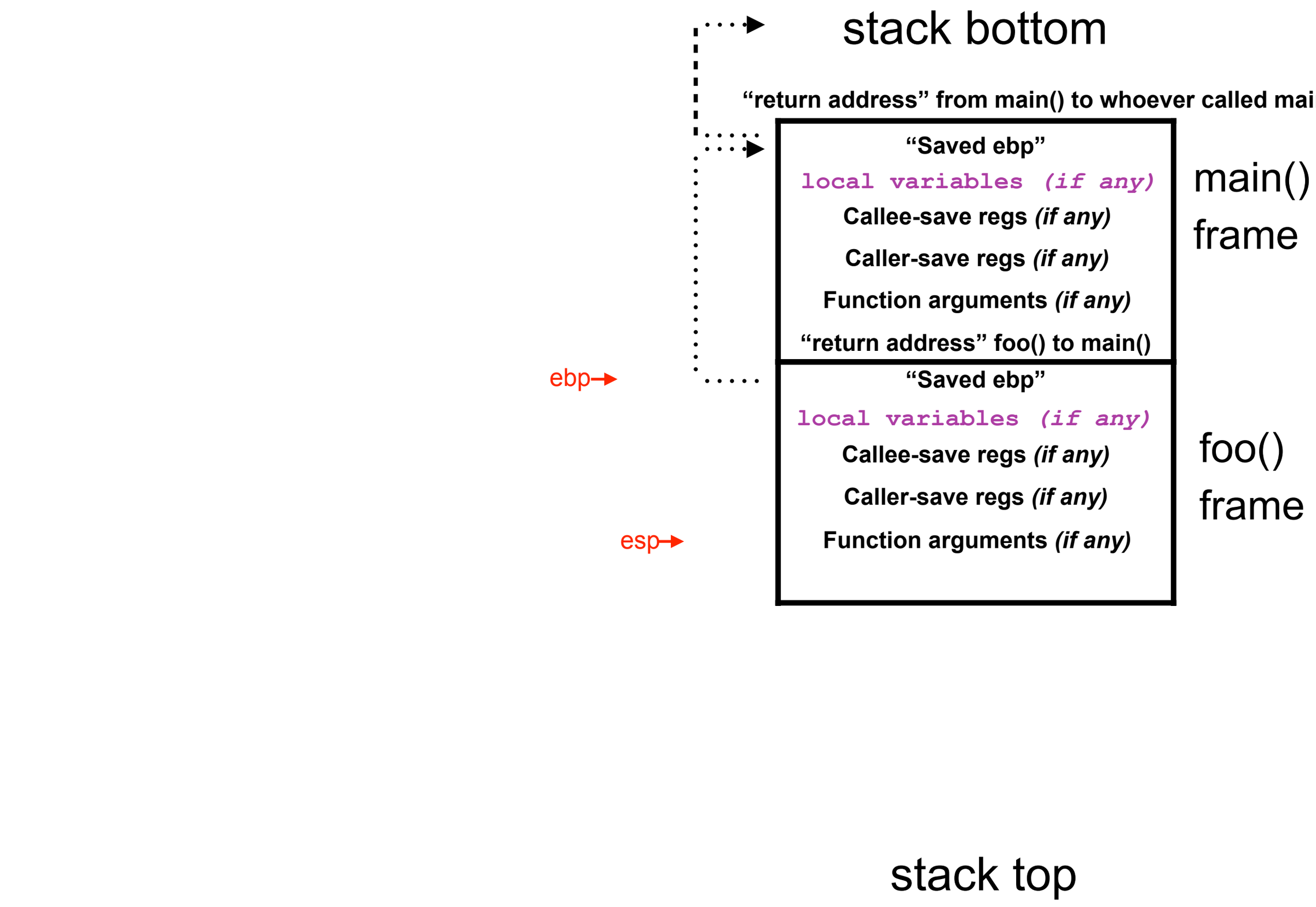
32-bit Maximum Stack Diagram



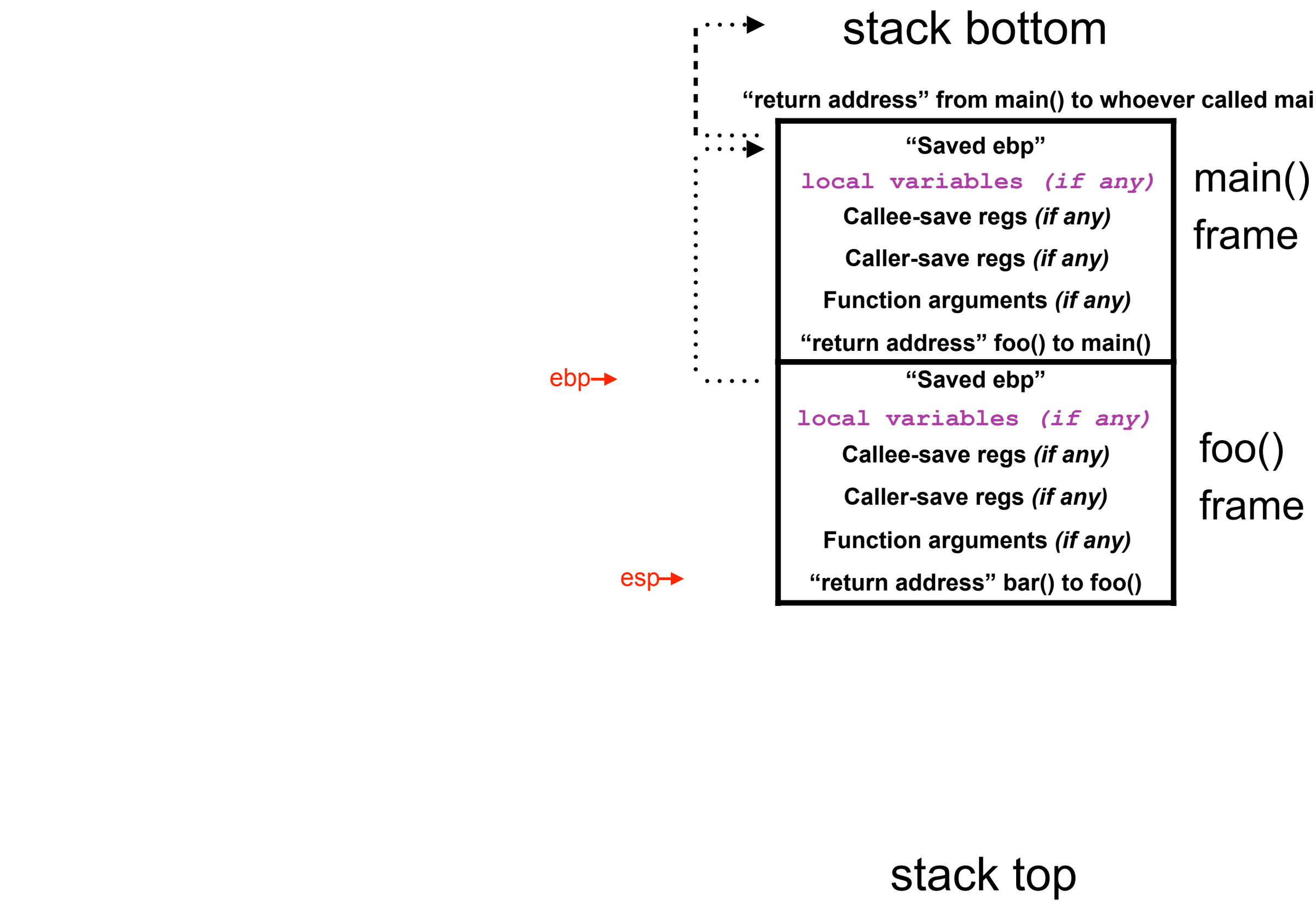
32-bit Maximum Stack Diagram



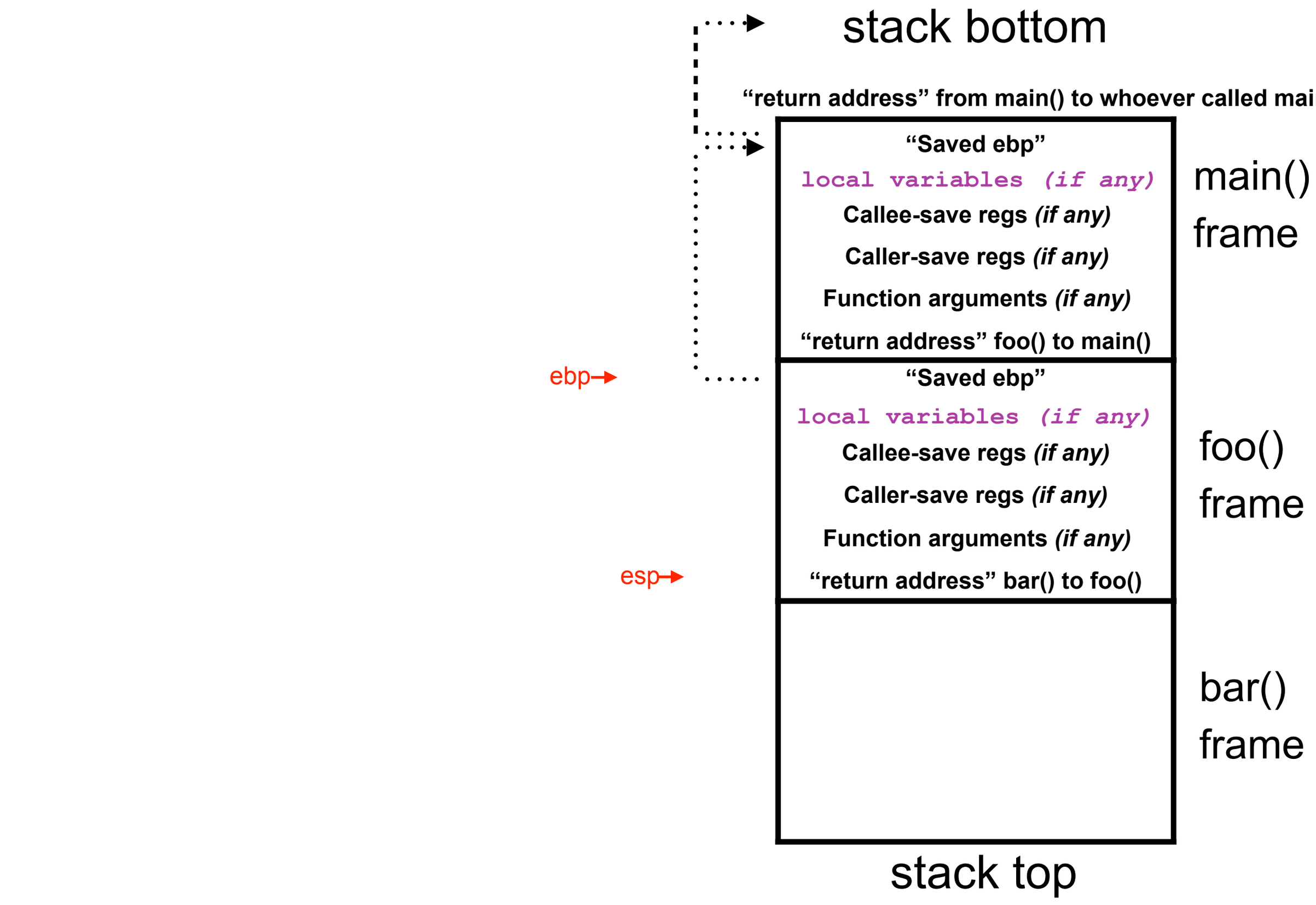
32-bit Maximum Stack Diagram



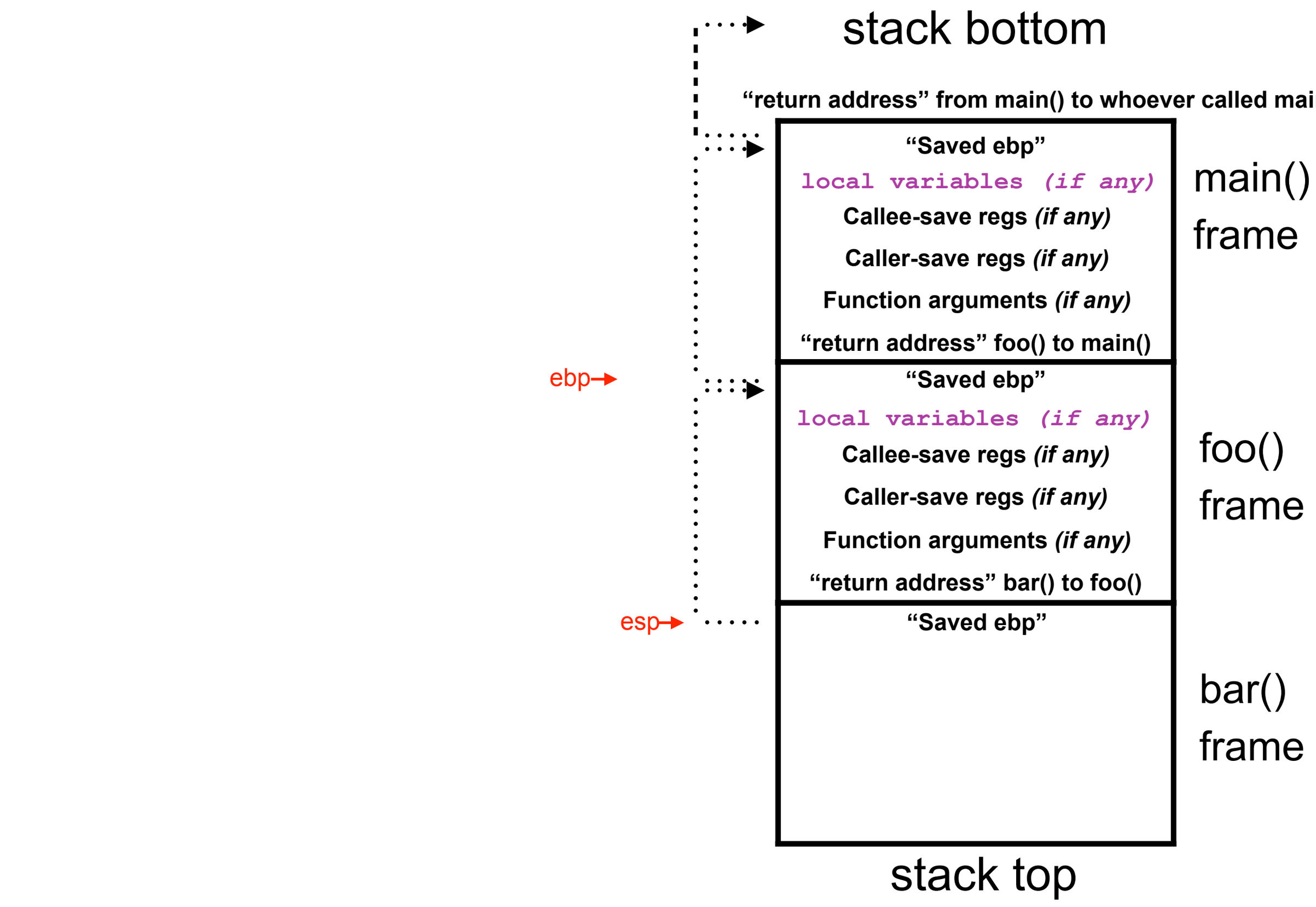
32-bit Maximum Stack Diagram



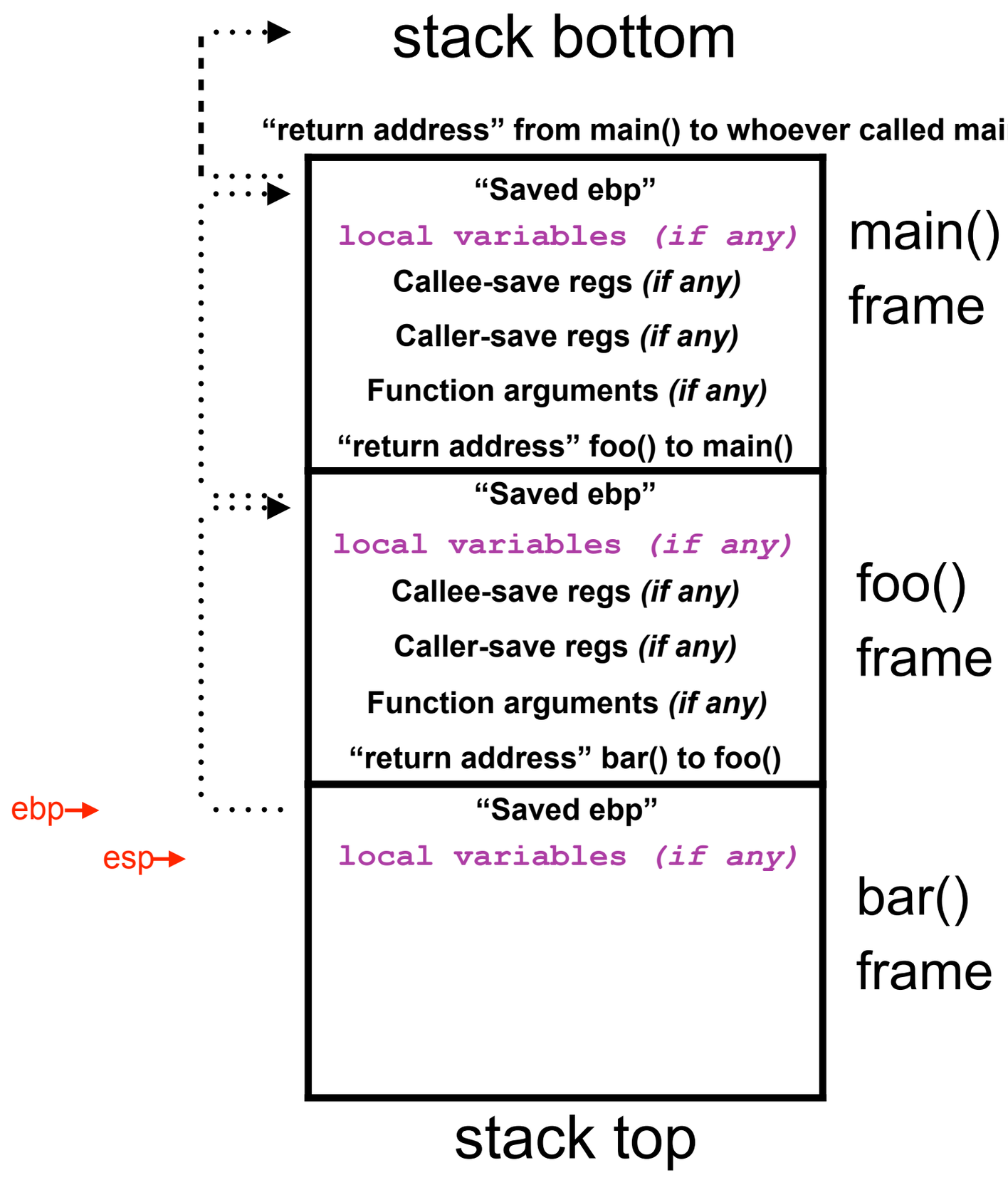
32-bit Maximum Stack Diagram



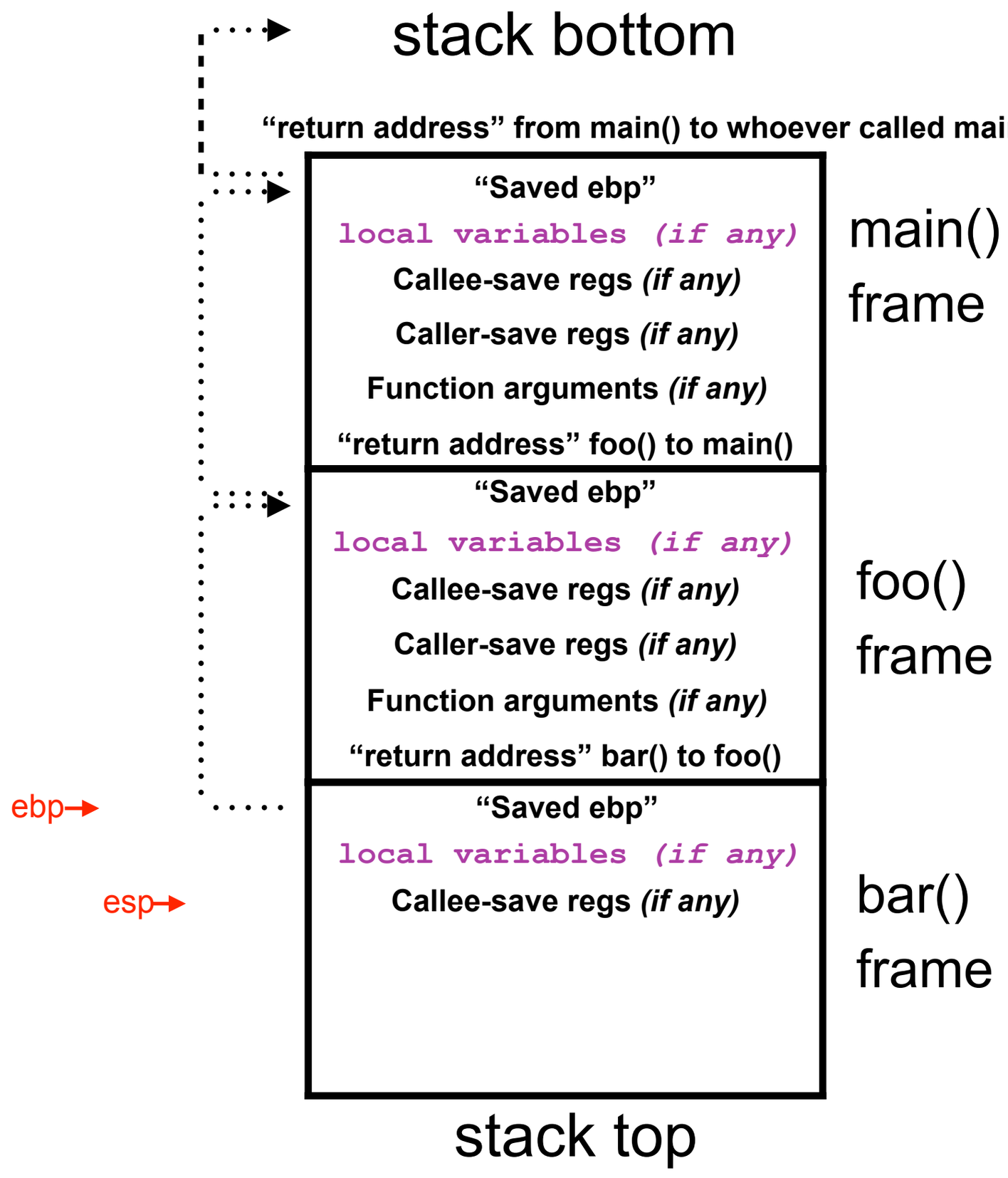
32-bit Maximum Stack Diagram



32-bit Maximum Stack Diagram



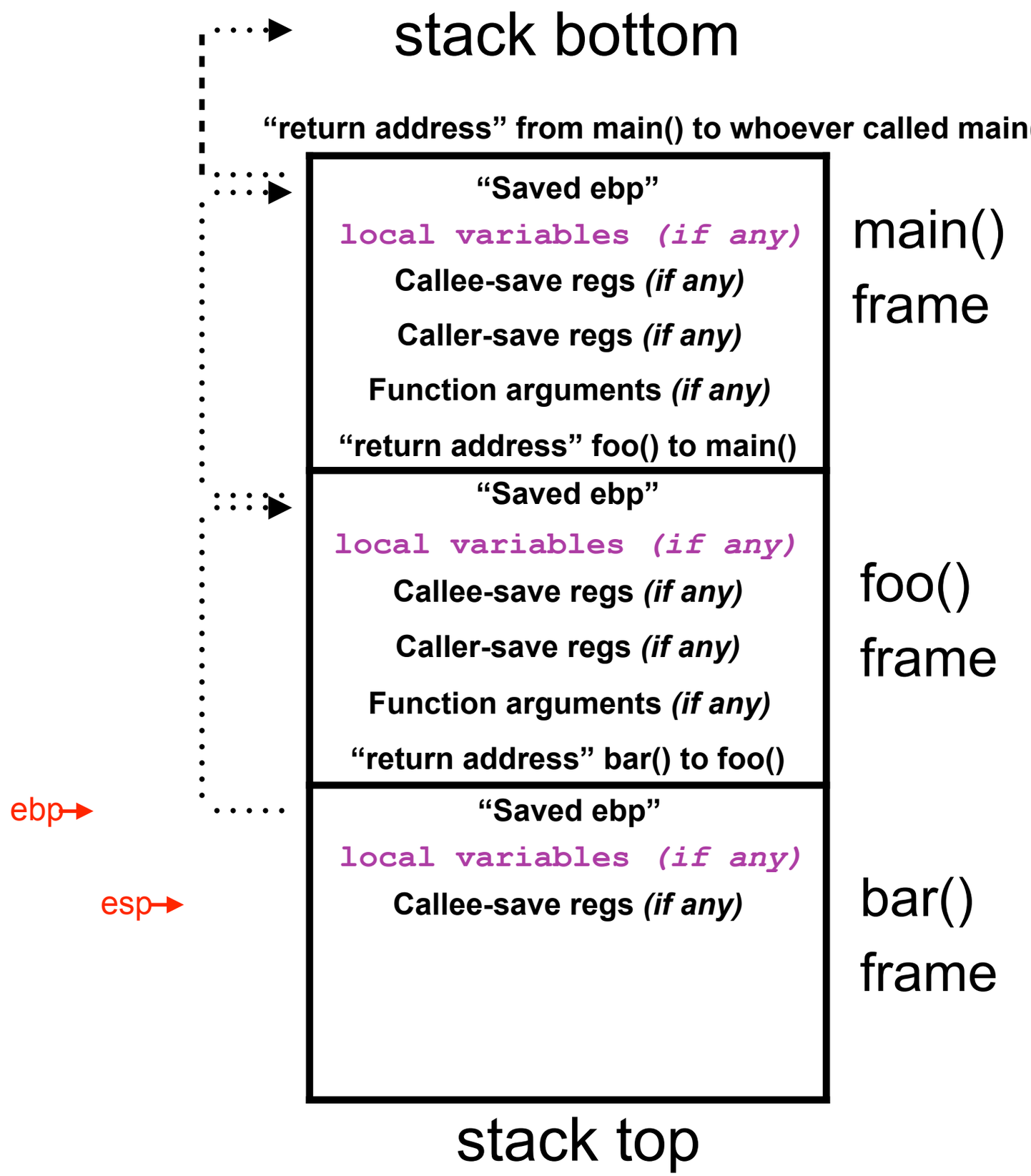
32-bit Maximum Stack Diagram



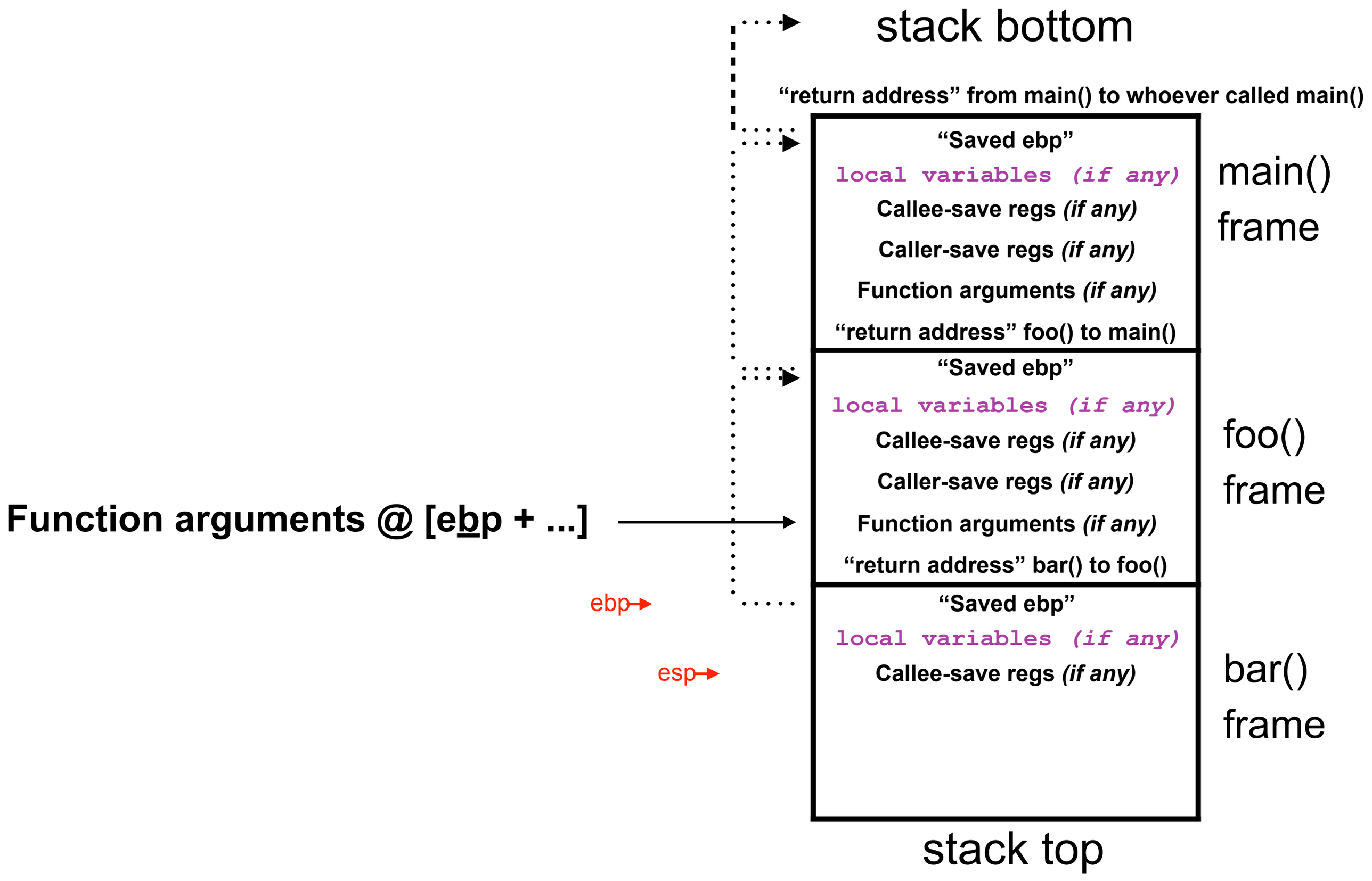
32-bit Stack Calling Conventions throwback 3

- *Function parameters* tend to get referenced as "ebp+offset"
- *Local variables* tend to get referenced as "ebp-offset"
  - Although gcc sometimes references as esp+offset

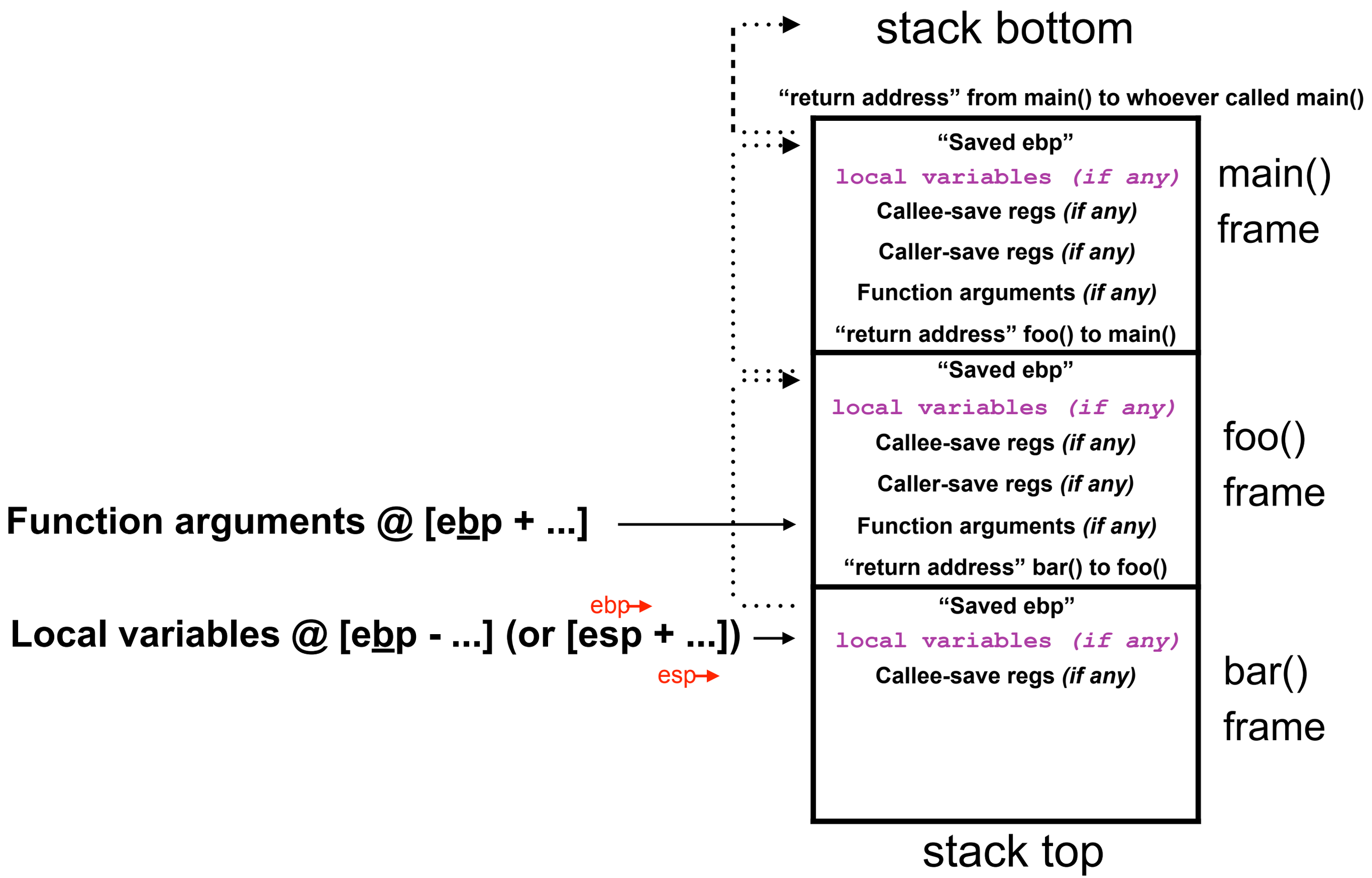
32-bit Maximum Stack Diagram 2



32-bit Maximum Stack Diagram 2



32-bit Maximum Stack Diagram 2



32-bit Stack Calling Conventions throwback 4

- MS also supports other conventions such as "fastcall" which passes first 2 params in EDX, ECX, and then the rest on the stack
  - It's like a more limited version of the x86-64 calling convention
  - No use of stack frames (since that's extra overhead, which would make it less fast!)

https://docs.microsoft.com/en-us/cpp/cpp/fastcall?view=msvc-160

32-bit VS vs. GCC et al.
Caller vs. Callee-save Regs

VS Caller-saved	EAX		ECX **(1) fastcall**	EDX **(2) fastcall**			
VS Callee-saved		EBX			ESI	EDI	EBP
System V (GCC et al.) Caller-saved	EAX		ECX	EDX	ESI	EDI	
System V (GCC et al.) Callee-saved		EBX					EBP

32-bit Stack Calling Conventions throwback 4

- TODO: thiscall

https://docs.microsoft.com/en-us/cpp/cpp/thiscall?view=msvc-160